EXP.NO:01 **DATE:31/01/25**

STUDY OF STAR UML

AIM:

INTRODUCTION:

Star UML is an open-source software modelling tool that supports the UML (Unified Modelling Language) framework for system and software modelling. It is based on UML version 1.4, provides eleven different types of diagrams, and accepts UML 2.0 notation, It actively supports the MDA (Model Driven Architecture) approach by supporting the UML profile concept and allowing to generate code for multiple languages.

SYSTEM REQUIREMENTS:

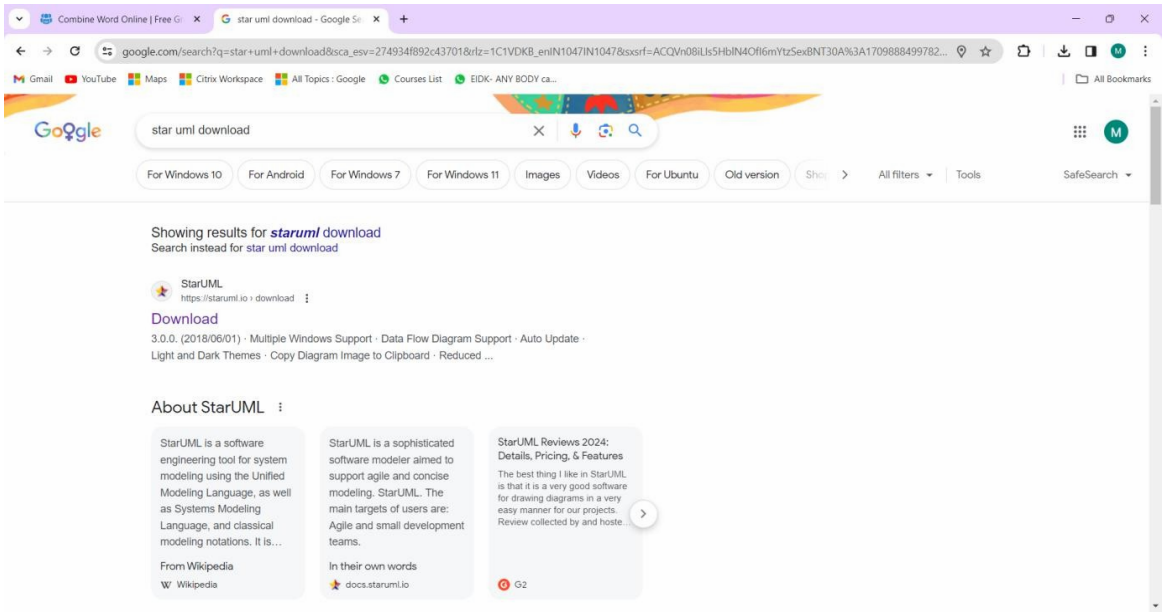
Windows 2000, Windows XP, or higher; Microsoft Internet Explorer 5.0 or higher; 128 MB RAM(256MB recommended); 110MB hard disc space (150MB space recommended).

INSTALLATION:

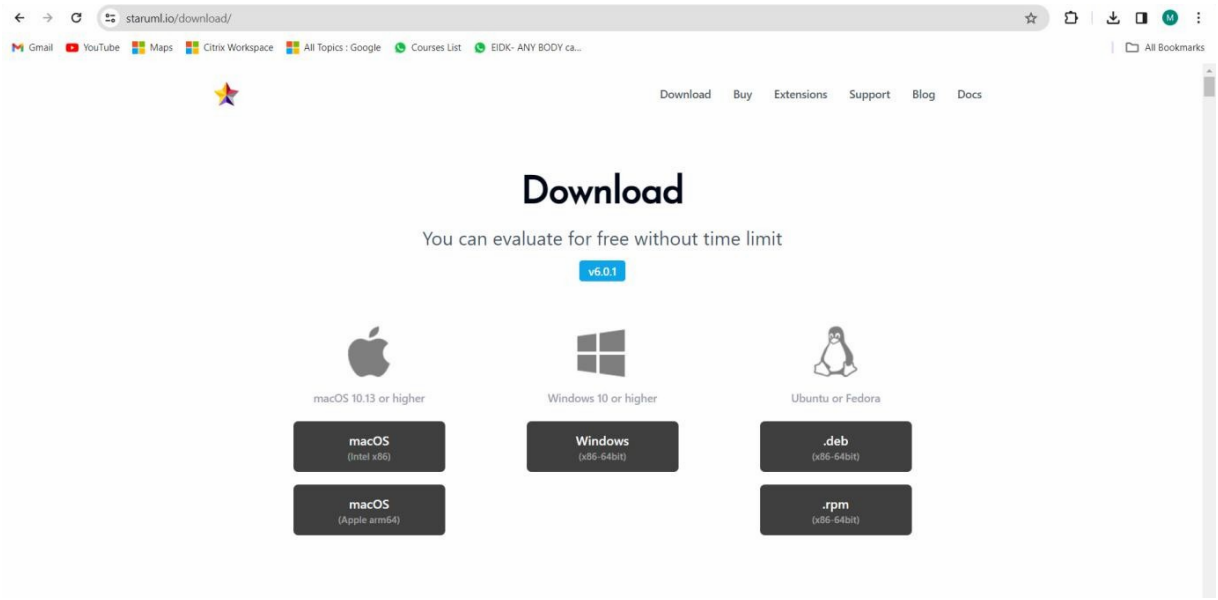
The installer follows the classic Windows install procedure without issues.

STEPS FOR DOWNLOADING STAR UML:

STEP1: Search for Star UML in Google.



STEP 2 : Download for Mac OS/Windows/.deb, .rpm.



STEP3: Install it



DOCUMENTATION:

The same help that could be browsed on the StarUML website is available with the tool on your desktop Documentation describes the concepts of the tool but on high-level vision. More detailed documentation is available for the diagramming functions. Sample projects are provided with the tool and one of them contains the model of the tool itself, showing that the developers were able to eat their dog food. Besides English, documentation exists in Korean, Japanese, and Russian.

CONFIGURATION:

Some general and diagram configuration options are available from the Tools/Options menu. You will find in this window also the configuration switches for the code generation. The interface is also very configurable as you can select what part of the tool you would like to view or not.

FEATURES:

When you start a new project, StarUML proposes which approach you want to use:4+1 (Kruchten).Rational, UML, components (from Cheesman and Daniels book), default or empty. Depending on the approach, profiles and/or frameworks may be included and loaded. If you don't follow a specific approach, the "empty" choice could be used.

StarUML makes a clear conceptual distinction between models, views, and diagrams. A Model is an element that contains information for a software model. A View is a visual expression of the information contained in a model, and a Diagram is a collection of view elements that represent the user's specific design thoughts.

StarUML is built as a modular and open tool. It provides frameworks for extending the functionality of the tool. It is designed to allow access to all functions of the model/meta-model and tool through COM Automation, and it provides an extension of menu and option items. Also, users can create their own approaches and frameworks according to their methodologies. The tool can also be integrated with any external tools.

1)Use Case Diagram

2)Class Diagram

3) Sequence Diagram

4)Collaboration Diagram

5) State chart Diagram

6) Activity Diagram

7)Component Diagram

8)Deployment Diagram

9)Composite Structure Diagram.

RESULT:

EXP.NO:02 **DATE:14/02/25**

STUDENT RESULT MANAGEMENT

AIM:

PROBLEM ANALYSIS AND PROJECT PLANNING

INTRODUCTION:

Student mark analysing system has been designed to carry out the mark analysis process in an educational institution. The results of respective departments can be efficiently computed without much manual involvement.

OBJECTIVES:

The purpose of this project is to define the requirements of the mark analysis system. This system reduces manual work to a great extent. The mark analysis is carried out by the system in an efficient manner.

SCOPE:

This system is very essential for every educational institution as it reduces manpower. This system can be used by all kinds of educational institutions to evaluate and analyse the marks and generate reports of specified criteria.

PROBLEM STATEMENT:

For analysing the marks obtained by students in an educational institution. We are tasked to build up a student mark analysing system. This is done to replace the manual entering and processing of marks which are error-prone and tedious. The system will have a Windows-based desktop interface to allow the faculty to enter marks obtained by the students, update them and generate various reports.

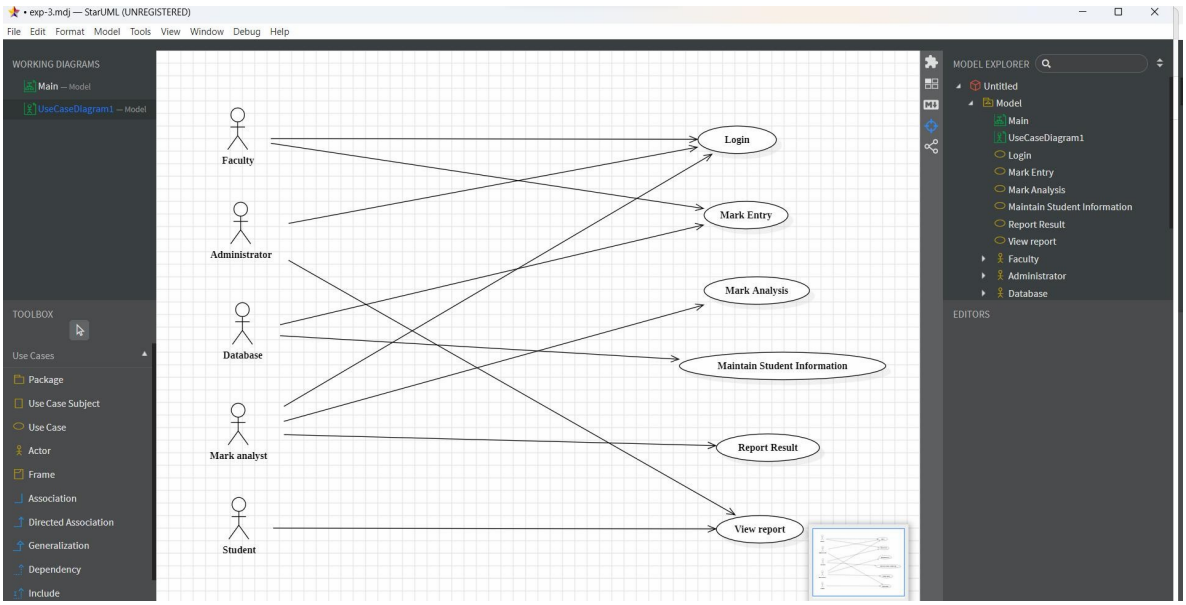
For security reasons, the administrator and faculty only can update the marks and other information. First, the user needs to login into to the system for accessing it. The system will

retain information on all the students and the institution. The system analyses the marks and generates the result reports. The marks and information about the students are stored in a database and the system works with the database. The faculty can enter the marks and student information through a visual environment. The updated details are stored in the database.

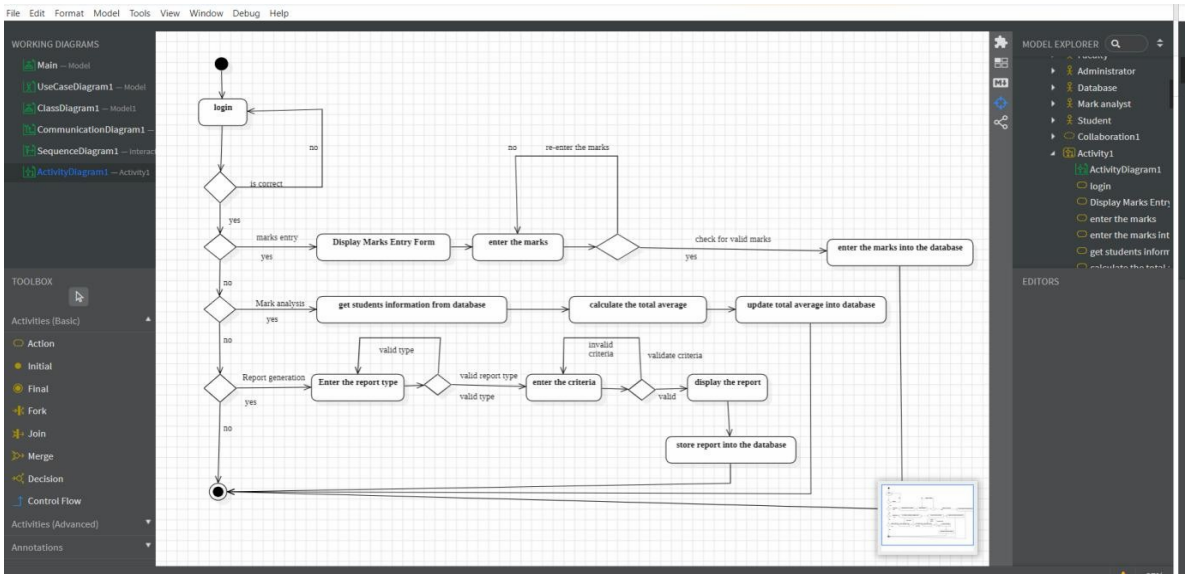
The system generates the overall result by analysing the marks. Mark analyser monitors this process. One of the most important features of the system is creating reports based on the given criteria. The user can create the following reports.

STAR UML DIAGRAMS:

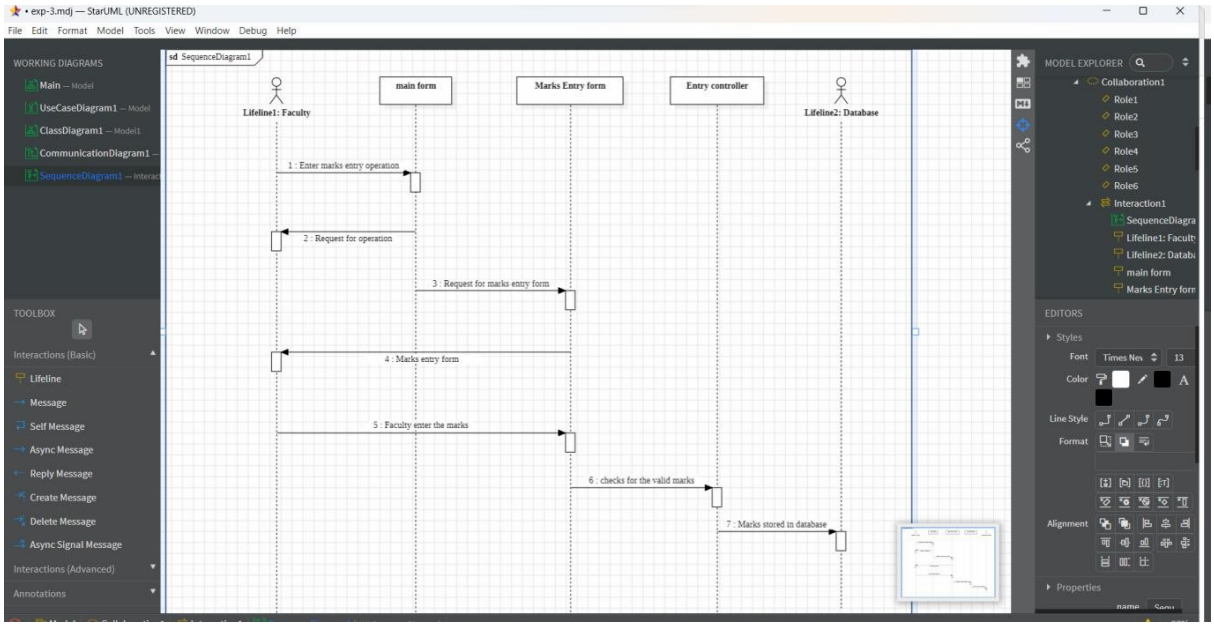
USE CASE DIAGRAM:



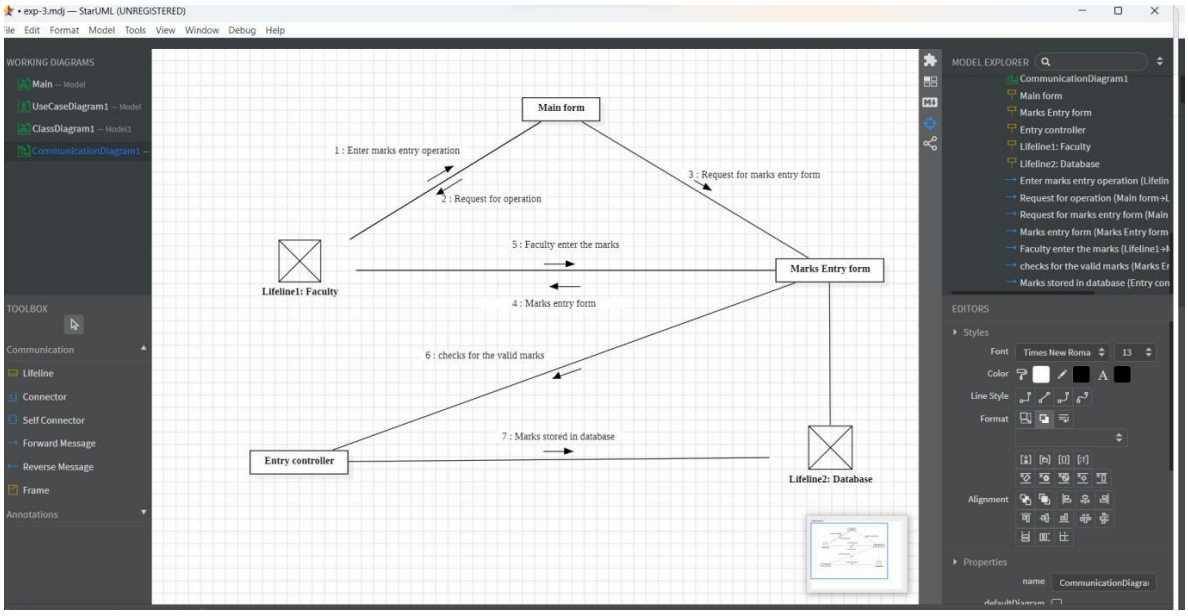
ACTIVITY DIAGRAM:



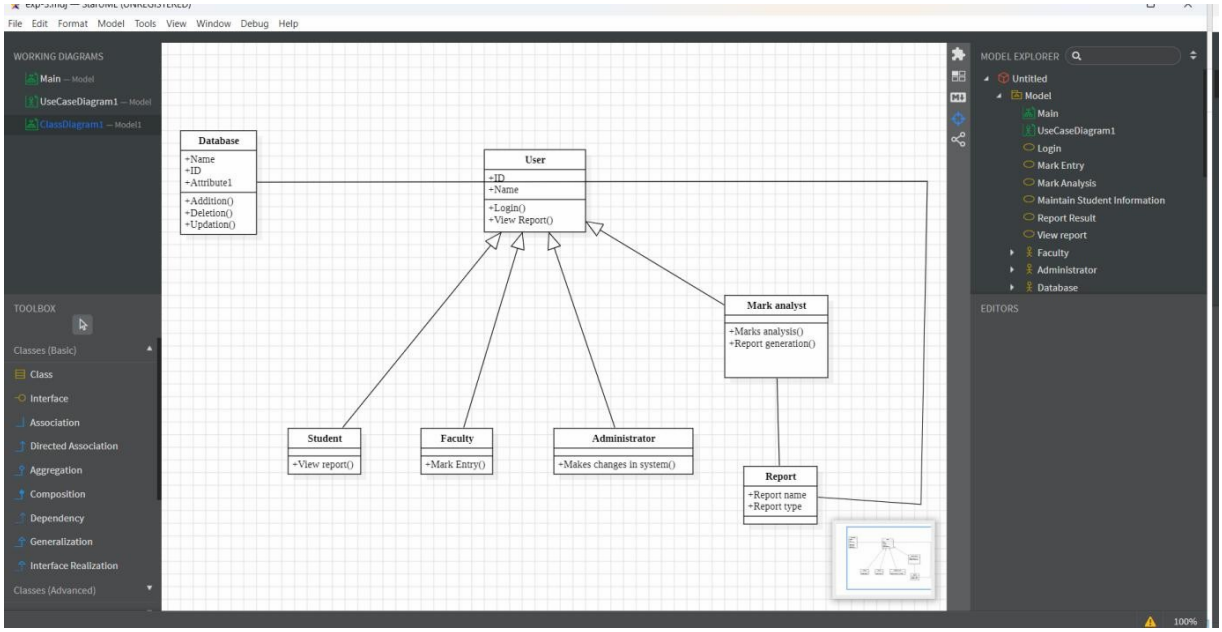
SEQUENCE DIAGRAM:



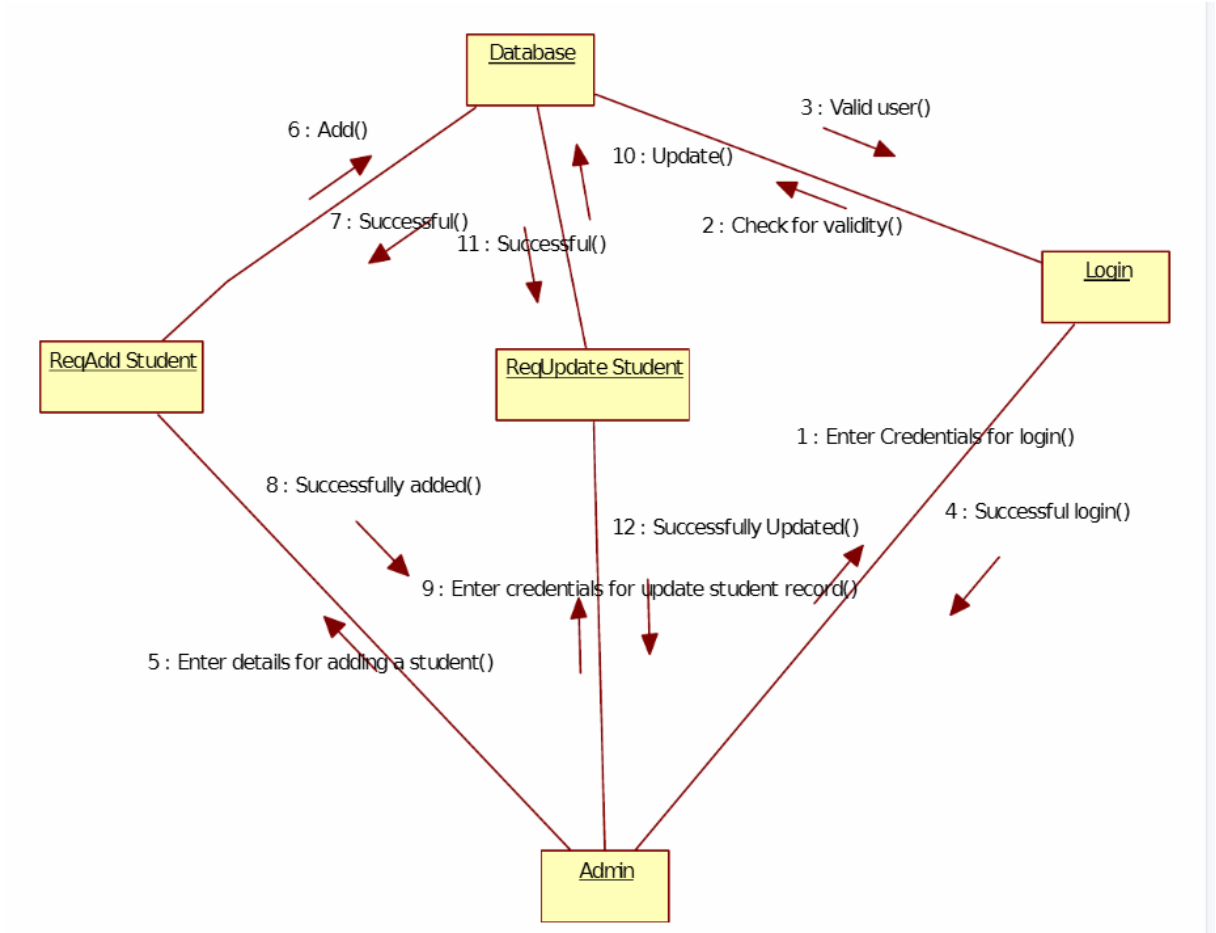
COMMUNICATION DIAGRAM:



CLASS DIAGRAM:



COLLABORATION DIAGRAM:



ALGORITHM:

1. Draw the class diagram and then save it.

2. Open the saved file with star uml.

3. Then go to tools at right corner.

4. Select the code language which we can prefer.

5. Then we have the code generator in tools so give access to it.

6. Then go to tools and select the language then it will take us to another window.

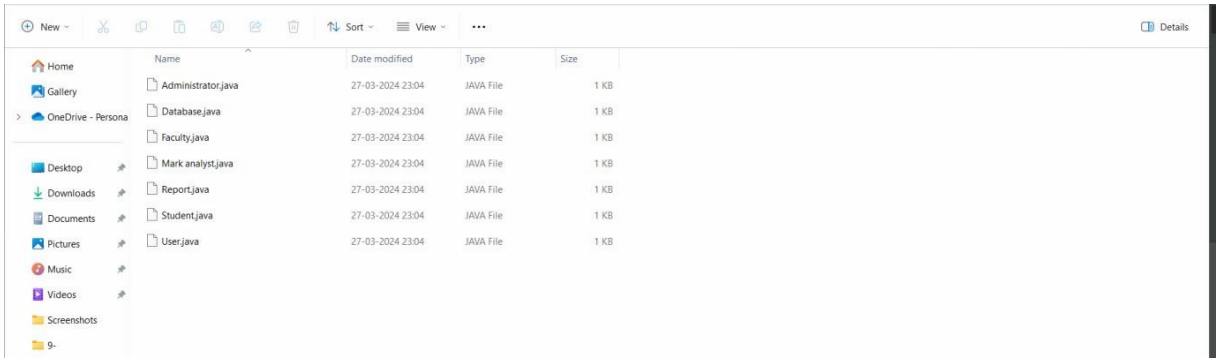
7. There select the class attribute to generate the code.

8. Then it will ask for the directory to store the code.

9. Select the directory and click on generate code.

10. Then code generated successfully.

OUTPUT:



ADMINISTRATOR:

```
import java.util.*;

/**

*

*/

public class administrator extends user {

    /**

    * Default constructor

    */

   public administrator() {

   }

    /**

    *

    */

   public void makes changes in systems() {

       // TODO implement here

   }
```

```
}
```

DATABASE:

```
import java.util.*;
/**
 *
 */
public class database {
    /**
     * Default constructor
     */
    public database() {
    }
    /**
     *
     */
    public void Name;
    /**
     *
     */
    public void id;
    /**
     *
     */
    public void addition() {
```

```
        // TODO implement here
}
    /**
     *
     */
    public void deletion() {
        // TODO implement here
    }
    /**
     *
     */
    public void updation() {
        // TODO implement here
    }
}
```

FACULTY:

```
import java.util.*;
/**
 *
 */
public class faculty extends user {
    /**
     * Default constructor
     */
```

```
    public faculty() {
}
     /**
     *
     */
    public void marks entry() {
        // TODO implement here
    }
}
```

MARK ANALYST:

```
import java.util.*;
/**
 *
 */
public class mark analyst extends user {
     /**
     * Default constructor
     */
    public mark analyst() {
    }
     /**
     *
     */
    public void mark analyst() {
```

```
        // TODO implement here
    }
    /**
     *
     */
    public void report generator() {
        // TODO implement here
    }
}
```

REPORT:

```
import java.util.*;
/**
 *
 */
public class report extends mark analyst {
      /**
     * Default constructor
     */
    public report() {
    }
     /**
     *
     */
    public void report name;
```

```
    /**
     *
     */
    public void report type;
}
```

STUDENT:

```
import java.util.*;
/**
 *
 */
public class student extends user {
     /**
     * Default constructor
     */
    public student() {
    }
     /**
     *
     */
    public void view report() {
        // TODO implement here
    }
}
```

USER:

```
import java.util.*;
/**
 *
 */
public class user1190 {
    /**
     * Default constructor
     */
    public user() {
    }
    public void login() {
        // TODO implement here
    }
    /**
     *
     */
    public void view report() {
        // TODO implement here
    }
}
```

RESULT:

EXP.NO:03 **DATE:21/02/25**

INVENTORY SYSTEM

AIM:

INTRODUCTION:

The stock maintenance system is basically for the customers who access the information about the stock (here it is books in the book store) and retrieves the information.

OBJECTIVES:

The purpose of the document is to define the requirements of the stock maintenance system. This supplementary specification lists the requirements that are not readily captured in the use cases of the use case model. The supplementary specification & the use case model together capture a complete set of requirement on the system.

SCOPE:

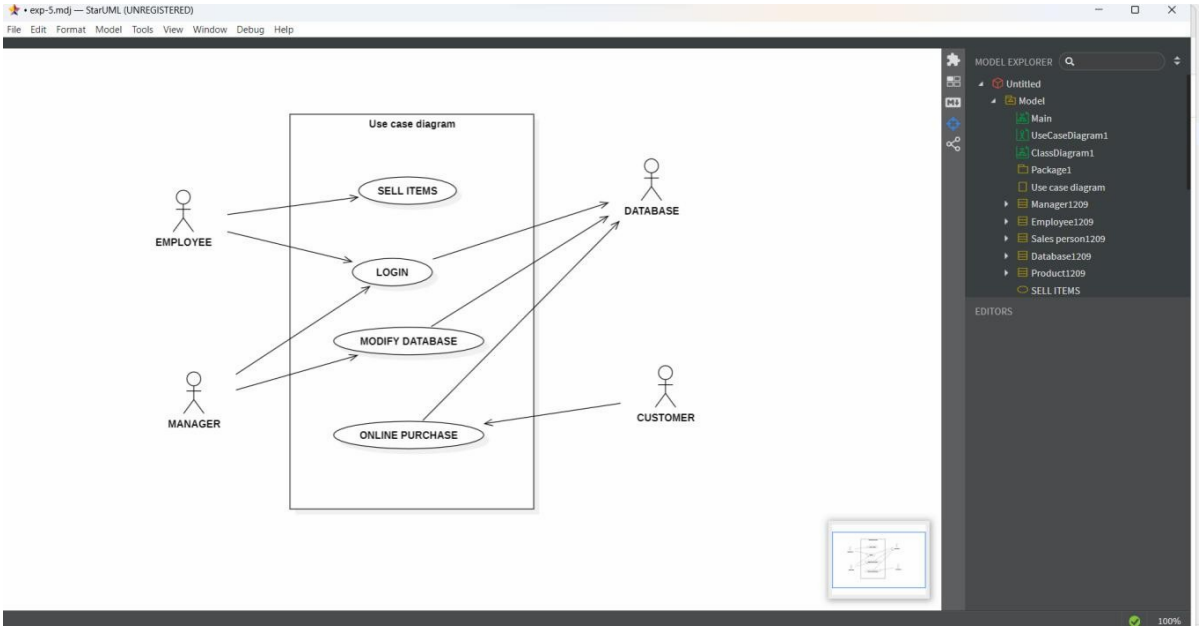
This supplementary specification applies to the stock maintenance system. This specification defines the non-functional requirements of the system, such as reliability, usability, performance and supportability as well as functional requirements that are common across a number of use cases.

PROBLEM STATEMENT:

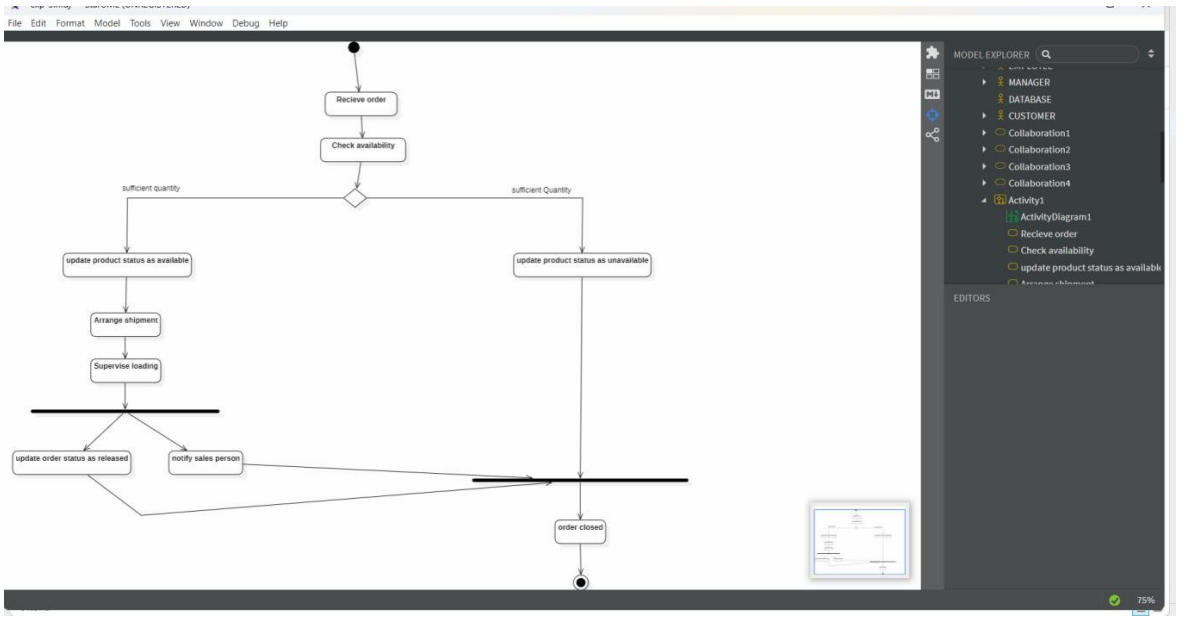
A new stock maintenance system for a book store is to replace the existing maintenance system which is inefficient. The new stock maintenance system will allow the employee to record information of the books available in the book store and generate report based on the total amount of sales. The new system will have a Windows based desktop interface to allow employees to enter the information of sale, purchase orders, change employee preferences and create reports. Employees can only access the information and purchase orders for security purpose. The system retains information on all the books in the shop. The system retains the records of the cost, edition, author, publication of the books. He can add the books at right time and update the database. The customer can view the availability of the required books and the price of the books. The customer can just view them but cannot make any changes.

STAR UML DIAGRAMS:

USE CASE DIAGRAM:

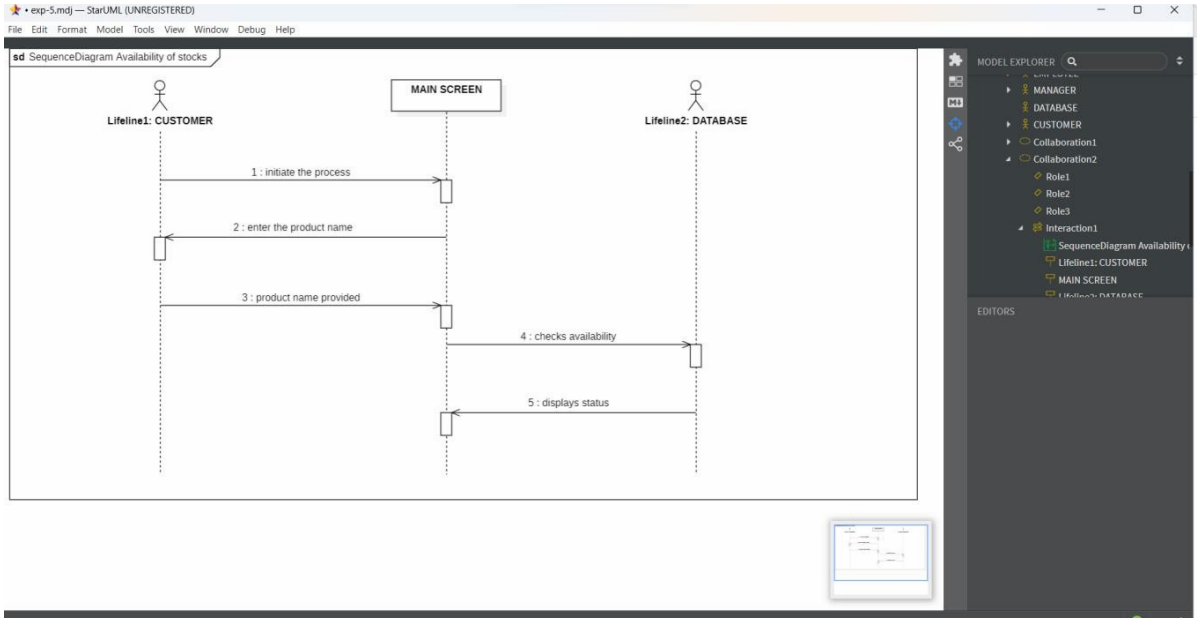


ACTIVITY DIAGRAM:

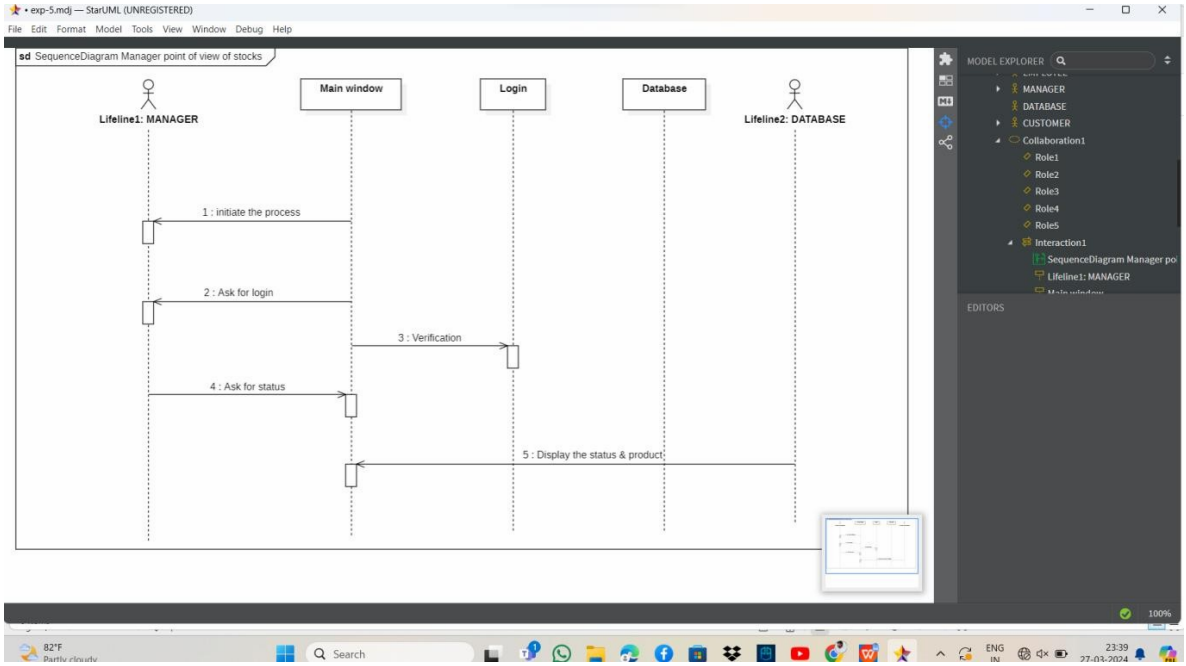


SEQUENCE DIAGRAM:

AVAILABILITY OF STOCKS

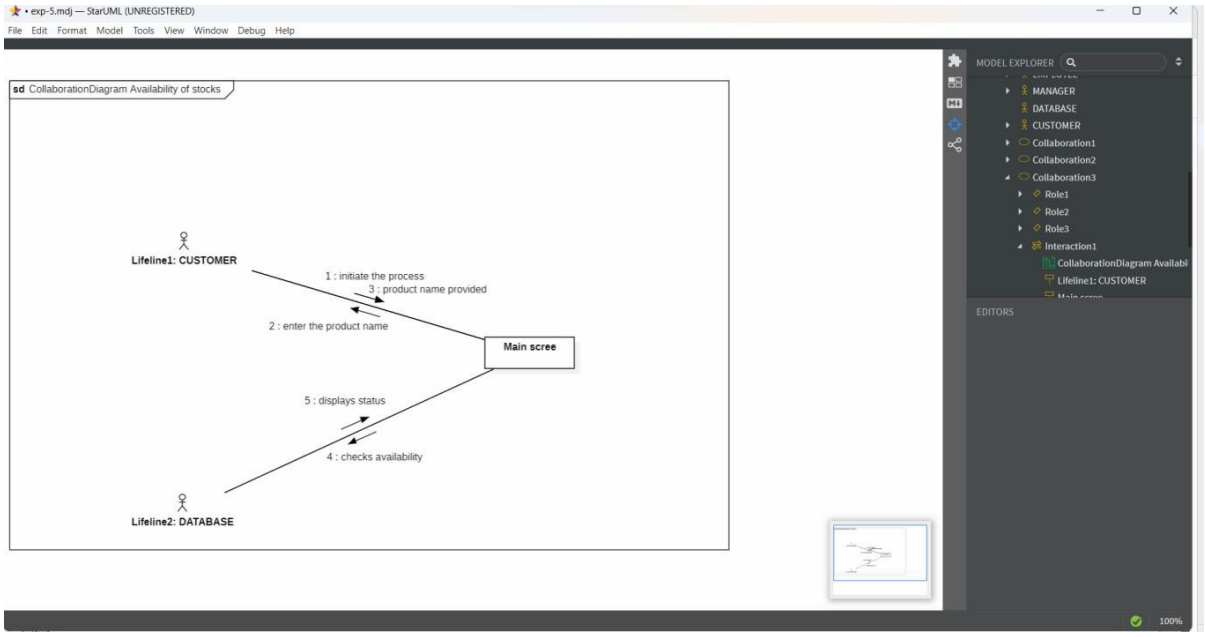


MANAGER POINT OF VIEW

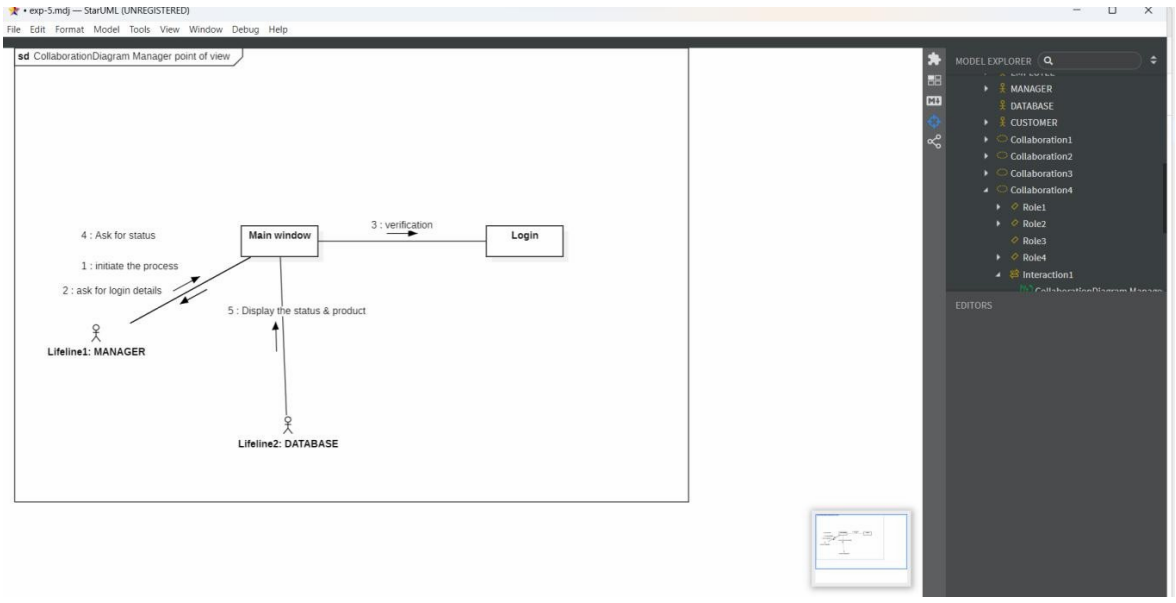


COMMUNICATION DIAGRAM:

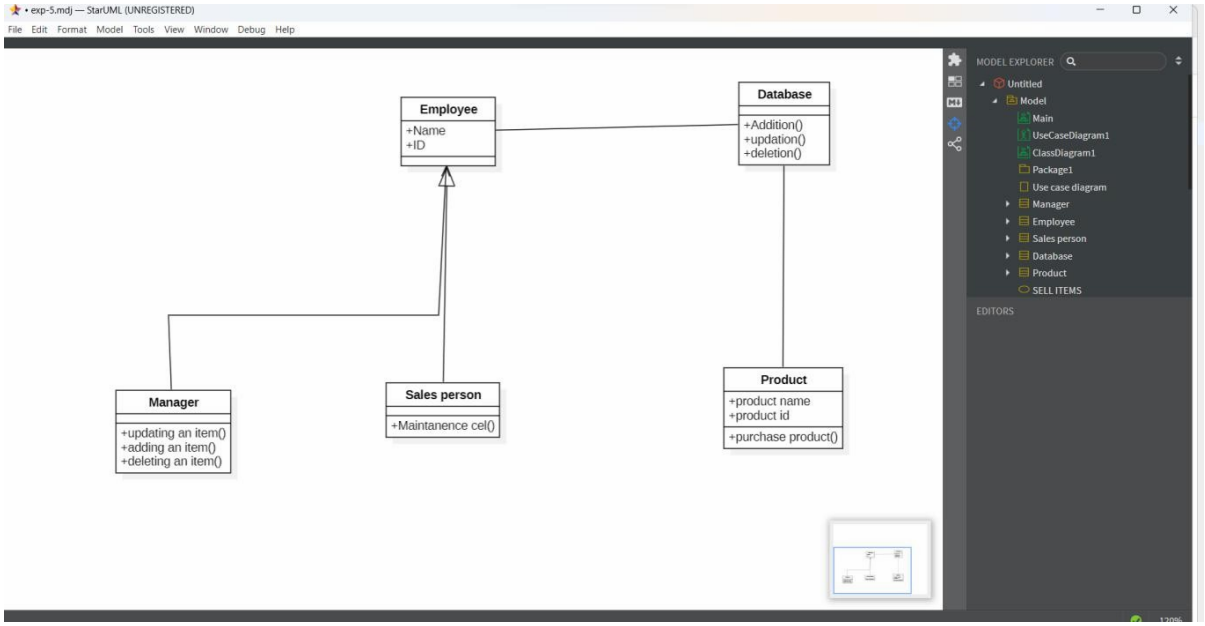
AVAILABILITY OF STOCKS



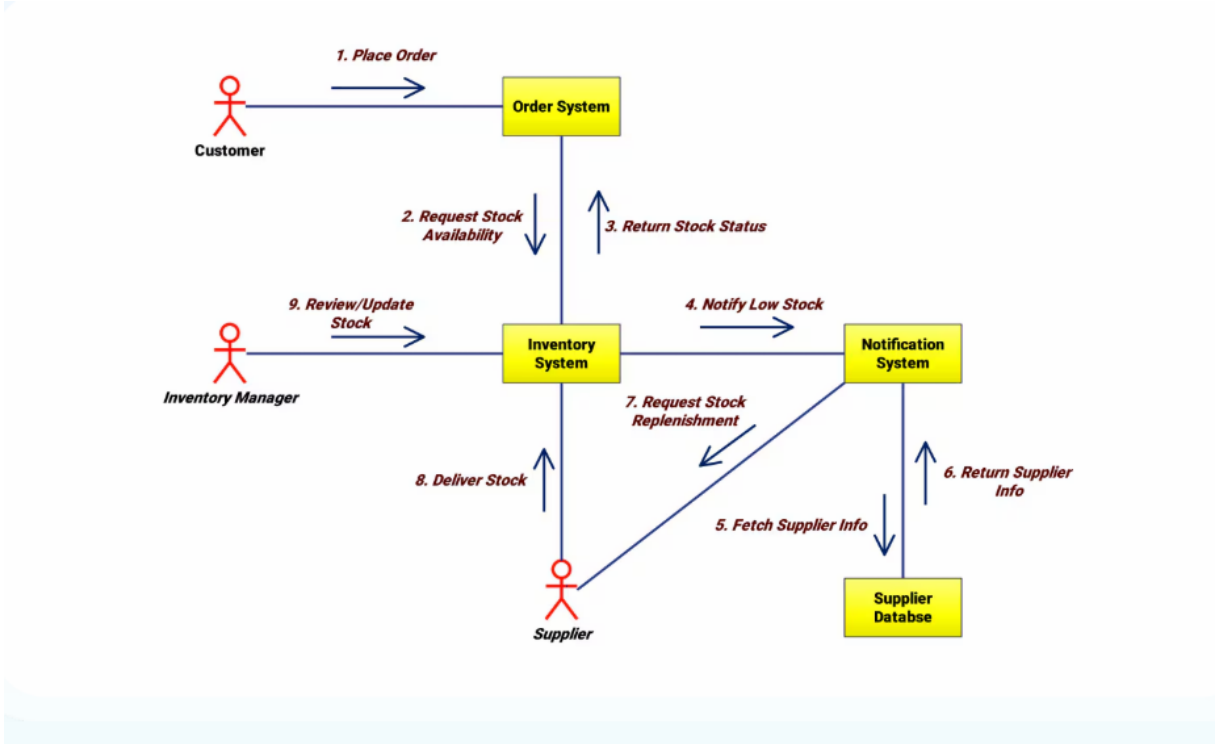
MANAGER POINT OF VIEW



CLASS DIAGRAM:



COLLOBARATION DIAGRAM:



ALGORITHM:

1. Draw the class diagram and then save it.

2. Open the saved file with star uml.

3. Then go to tools at right corner.

4. Select the code language which we can prefer.

5. Then we have the code generator in tools so give access to it.

6. Then go to tools and select the language then it will take us to another window.

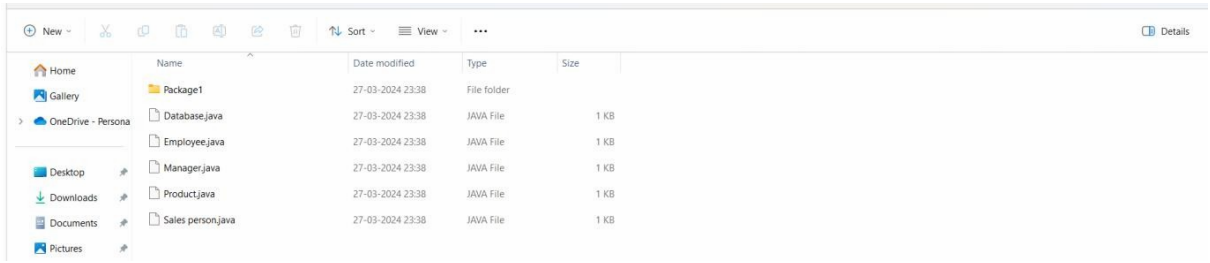
7. There select the class attribute to generate the code.

8. Then it will ask for the directory to store the code.

9. Select the directory and click on generate code.

10. Then code generated successfully.

OUTPUT:



DATA BASE:

```
import java.util.*;

public class Database  {

public Database () {

    }

    public void addition() {

        // TODO implement here

    }

    public void updation() {

        // TODO implement here

    }

    public void deletion() {

        // TODO implement here

    }

}
```

EMPLOYEE:

```
import java.util.*;
public class Employee  {
public Employee () {
   }
public void Name;
public void ID;
public void Operation1() {
       // TODO implement here
 } }
```

MANAGER:

```
import java.util.*;
public class Manager  {
public Manager () {
   }
public void Attribute1;
public void updating an item() {
       // TODO implement here
   }
 public void adding an item() {
       // TODO implement here
   }
 public void deleting an item()
  // TODO implement here   } }
```

PRODUCT:

```
import java.util.*; public class
Product  { public Product () {
    }
public void Product name; public void
purchase product() {
        // TODO implement here
    }
}
```

SALES PERSON:

```
import java.util.*;
public class Sales person  extends Employee  { public Sales
person () {
    }
public void Maintainence cell() {
        // TODO implement here
    }
}
```

RESULT:

EX.NO: 04 **DATE:28/02/25**

ONLINE RAILWAY RESERVATION SYSTEM

AIM:

PROBLEM ANALYSIS AND PROJECT PLANNING:

INTRODUCTION:

This document deals with online ticket reservation for railway. This document is designed in such a way that the reader understands it. The use case description and other documents are described in such a way that the system reaches the people easily.

OBJECTIVES:

The purpose of the document is to know about the availability of seats, railway etc. According to the requirements, passenger reserves his/her tickets.

SCOPE:

This document for online ticket reservation for airline makes the work easy for the passenger to book tickets.

PROBLEM STATEMENT:

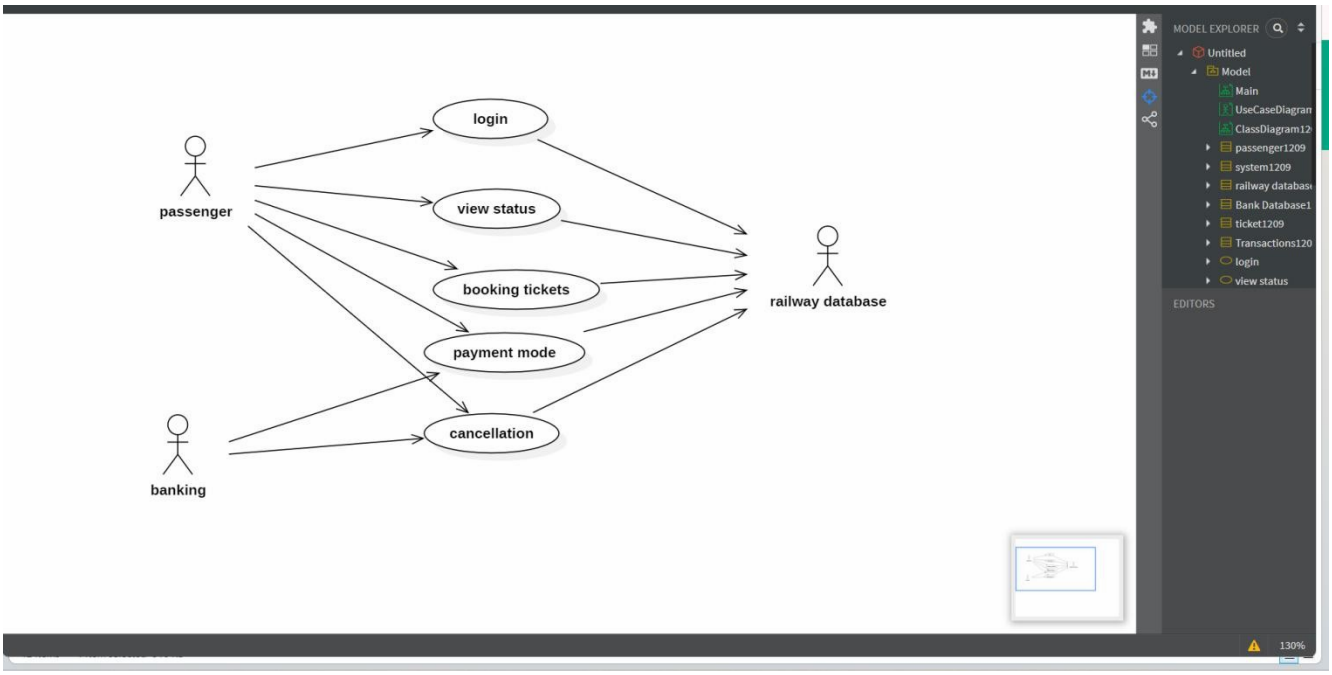
Computers are an integral part of our day today life. It makes the work easy and faster. Every job is computerized now. So is the ticket reservation, we can book our tickets online. During the reservation of tickets the passenger has to select the origin, date of journey, passport number, etc.

The reservation counter keeps track of the passenger's information. The system will have all the details about the trains and the facilities provided by them. There are various trains

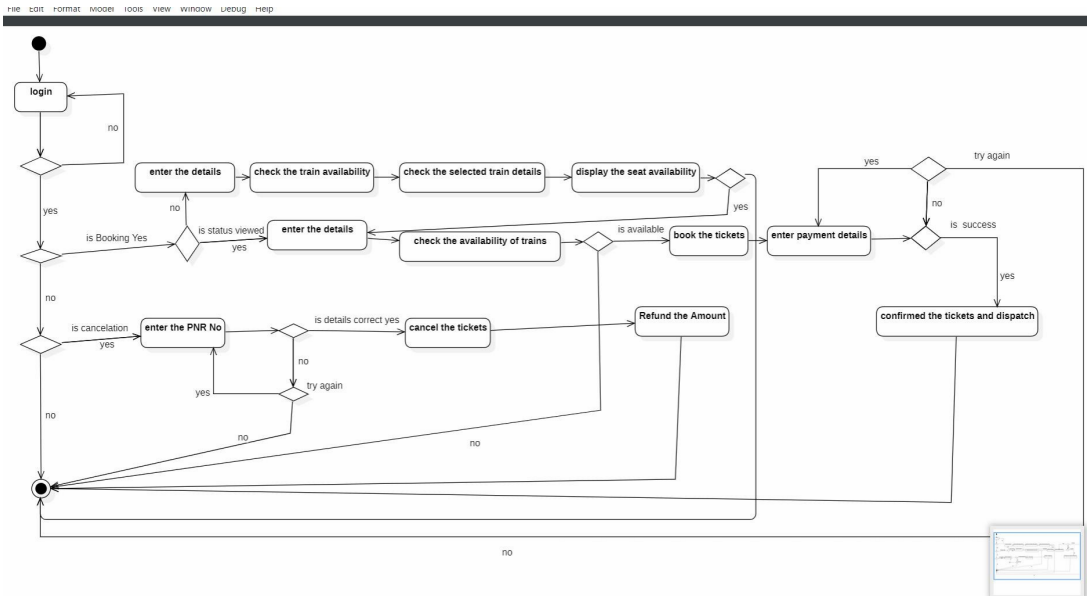
provided according to the convenience of the passenger. A database is maintained by the database administrator.

STAR UML DIAGRAMS:

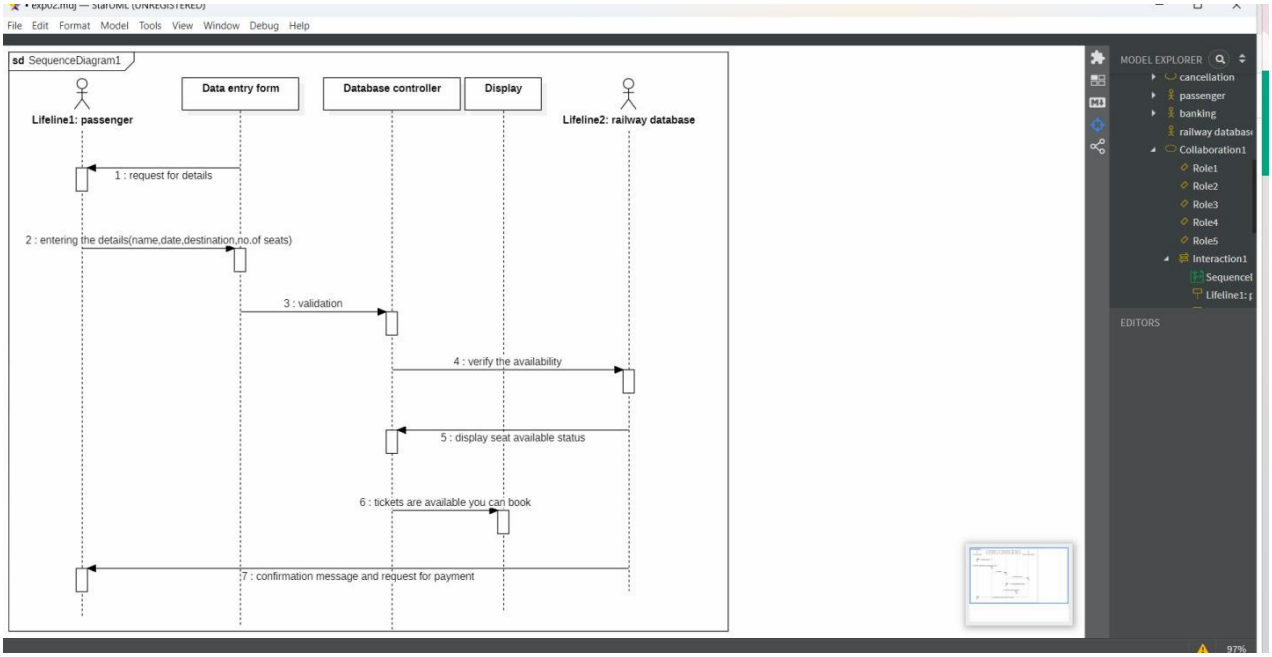
USE CASE DIAGRAM:



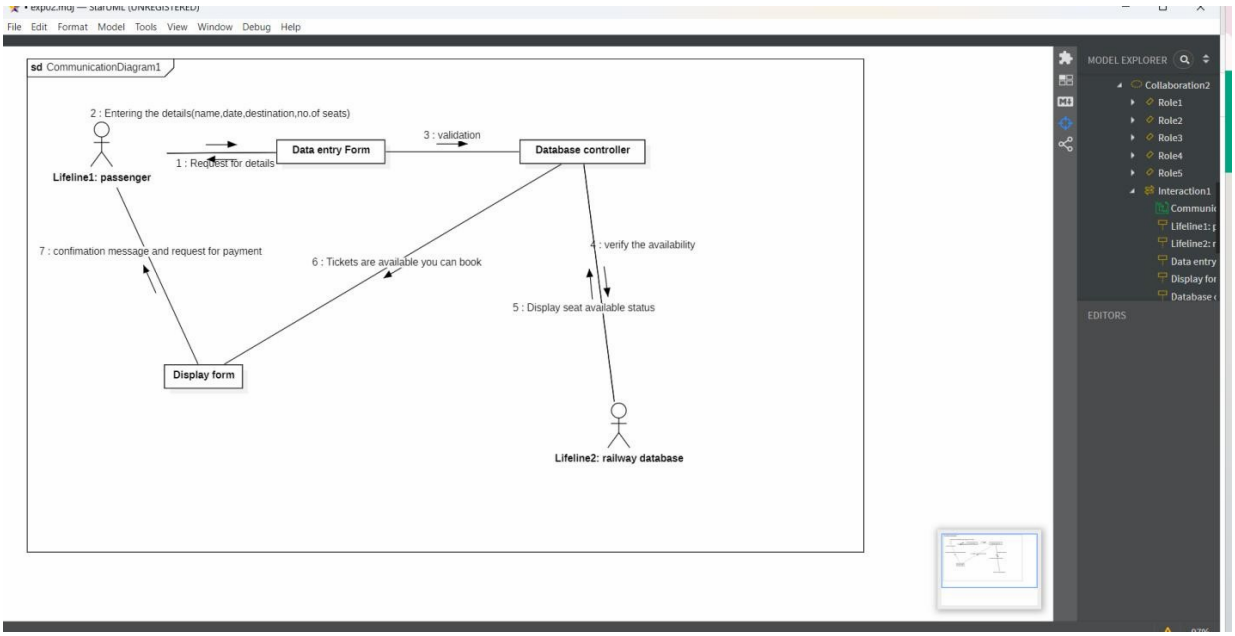
ACTIVITY DIAGRAM:



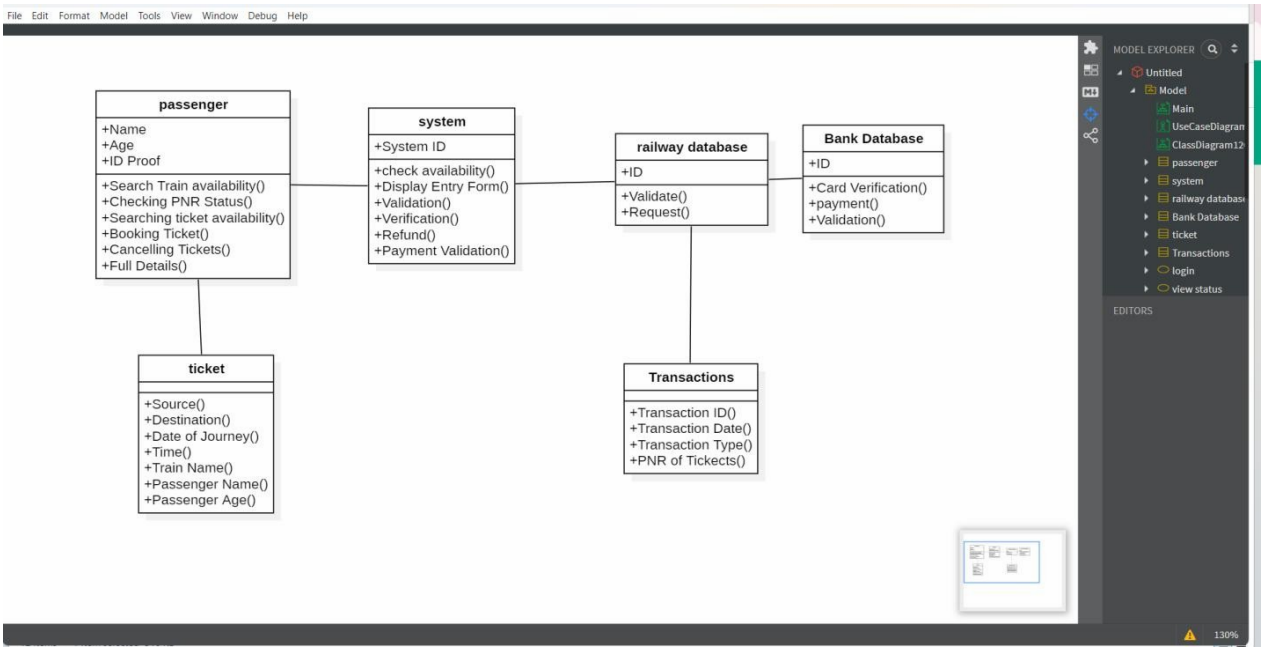
SEQUENCE DIAGRAM:



COMMUNICATION DIAGRAM:



CLASS DIAGRAM:



STEPS TO GENERATE CODE IN STAR UML:

) Draw the class diagram and then save it.

2) Open the saved file with star uml.

3) Then go to tools at right corner.

4) Select the code language which we can prefer.

5) Then we have the code generator in tools so give access to it.

6) Then go to tools and select the language then it will take us to another window.

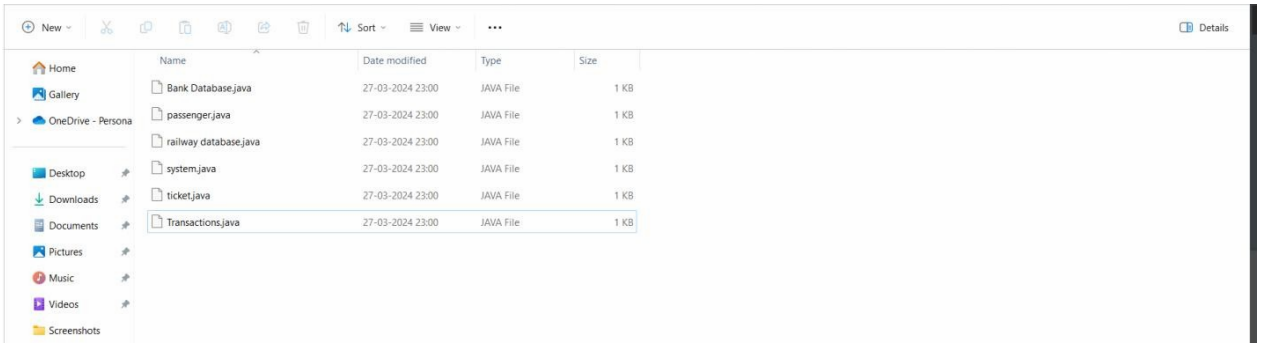
7) There, select the class attribute to generate the code.

8) Then it will ask for the directory to store the code.

9) Select the directory and click on generate code.

10) Then code generated successfully

OUT PUT:



PASSENGER:

```
import java.util.*;

public class passenger {

public passenger () {

    }

        public void Name;

        public void Age;

        public void ID Proof;

                public void Search Train availability() {

        // TODO implement here

    }

                public void Checking PNR Status() {

        // TODO implement here

    }

                public void Searching ticket availability() {
```

```
        // TODO implement here
    }
            public void Booking Ticket() {
        // TODO implement here
    }
            public void Cancelling Tickets() {
        // TODO implement here
    }
            public void Full Details() {
        // TODO implement here
    }
}
```

BANK DATABASE:

```
import java.util.*;
public class Bank Database  {
    public Bank Database () {
    }
    public void ID;
    public void Card Verification() {
        // TODO implement here
    }
```

```
    public void payment() {
        // TODO implement here
    }
    public void Validation() {
        // TODO implement here
    }
}
```

RAILWAY DATABASE:

```
import java.util.*;
public class railway database  {
    public railway database {
    }
    public void ID;
    public void Validate() {
        // TODO implement here
    }
    public void Request() {
        // TODO implement here
    }
}
```

SYSTEM:

```
import java.util.*;

public class system  {

    public system () {

    }

    public void System ID;

    public void check availability() {

        // TODO implement here

    }

    public void Display Entry Form() {

        // TODO implement here

    }

    public void Validation() {

        // TODO implement here

    }

     public void Verification() {

        // TODO implement here

    }

    public void Refund() {

        // TODO implement here

    }
```

```
    public void Payment Validation() {
        // TODO implement here
    }
}
```

TICKET:

```
import java.util.*;
public class ticket  {
    public ticket () {
    }
    public void Source() {
        // TODO implement here
    }
    public void Destination() {
        // TODO implement here
    }
    public void Date of Journey() {
        // TODO implement here
    }
    public void Time() {
        // TODO implement here
    }
```

```
    public void Train Name() {
        // TODO implement here
    }
    public void Passenger Name() {
        // TODO implement here
    }
    public void Passenger Age() {
        // TODO implement here
    }
}
```

TRANSACTIONS:

```
import java.util.*;
public class Transactions  {
    public Transactions () {
    }
    public void Transaction ID() {
        // TODO implement here
    }
    public void Transaction Date() {
        // TODO implement here
    }
```

```
    public void Transaction Type() {
        // TODO implement here
    }
    public void PNR of Tickects() {
        // TODO implement here
    }
}
```

RESULT:

EXP.NO:05 **DATE:07/03/25**

HOTEL MANAGEMENT SYSTEM

AIM:

INTRODUCTION:

To design a library management system using IBM Rational Rose. Implement this system using Visual. Basic Language with Oracle/SQL.

OBJECTIVE:

- To develop a robust hotel management system using object-oriented principles.
- To simplify hotel operations like room booking, billing, and customer management.
- To provide a user-friendly GUI for staff and admin.
- To ensure accurate and real-time updates to bookings and availability.

SCOPE:

The Hotel Management System will cover:

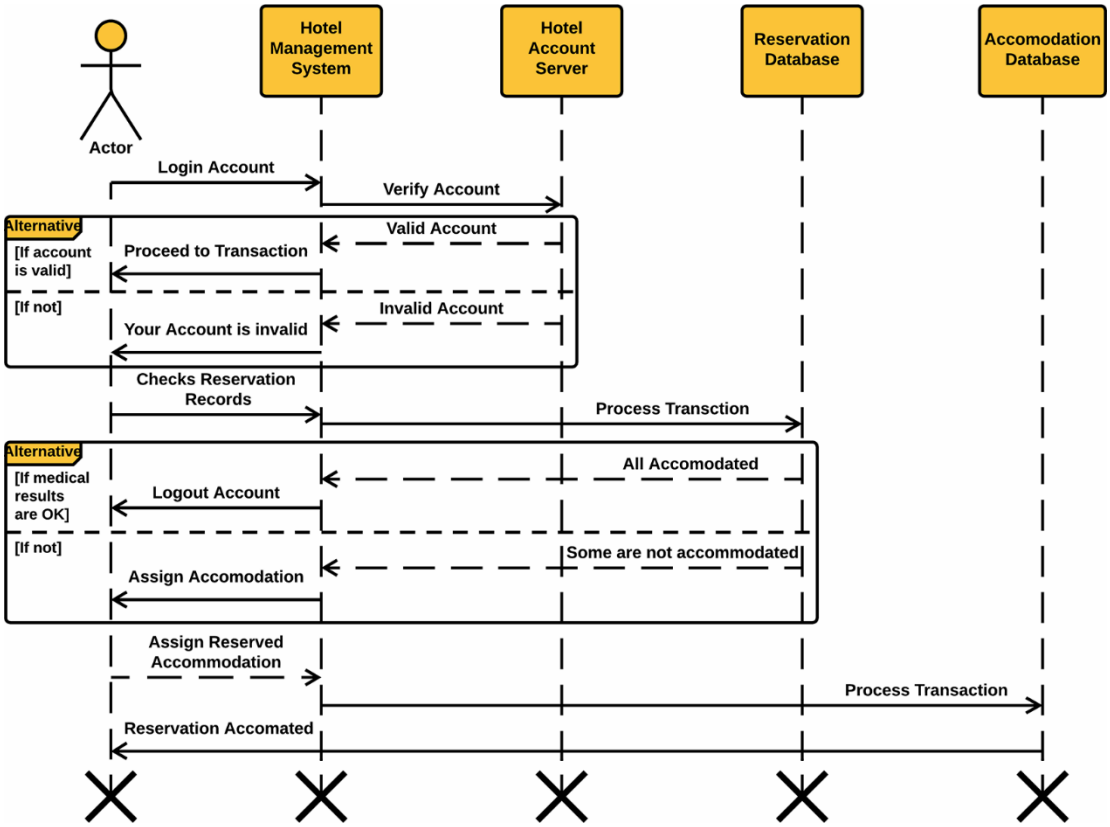
Room booking and cancellation, Customer ,check-in and check-out ,Room availability tracking, Payment and billing system ,Admin module for managing rooms and user dataAND Report generation (optional for advanced version)

PROBLEM STATEMENT:

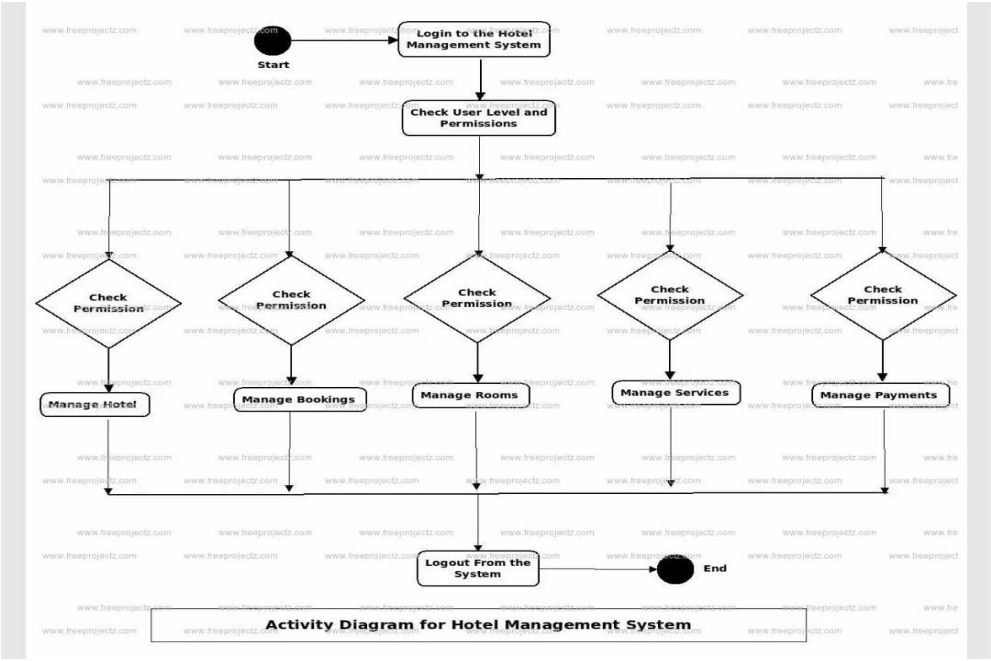
Manual hotel operations are time-consuming, prone to human errors, and inefficient in handling large numbers of guests. A digital Hotel Management System is needed to automate and streamline operations such as booking, billing, and room management, ensuring better customer service and staff efficiency. Hotel Management system should have these UML Diagrams, What are UML Diagrams, How to form them and how they worked. In this article you will also know the bonus diagrams that will help you in understanding how the Hotel Management System works. Each of these Diagrams comes with complete explanation to let you understand its contribution to the Hotel Management system.

UML DIAGRAM:

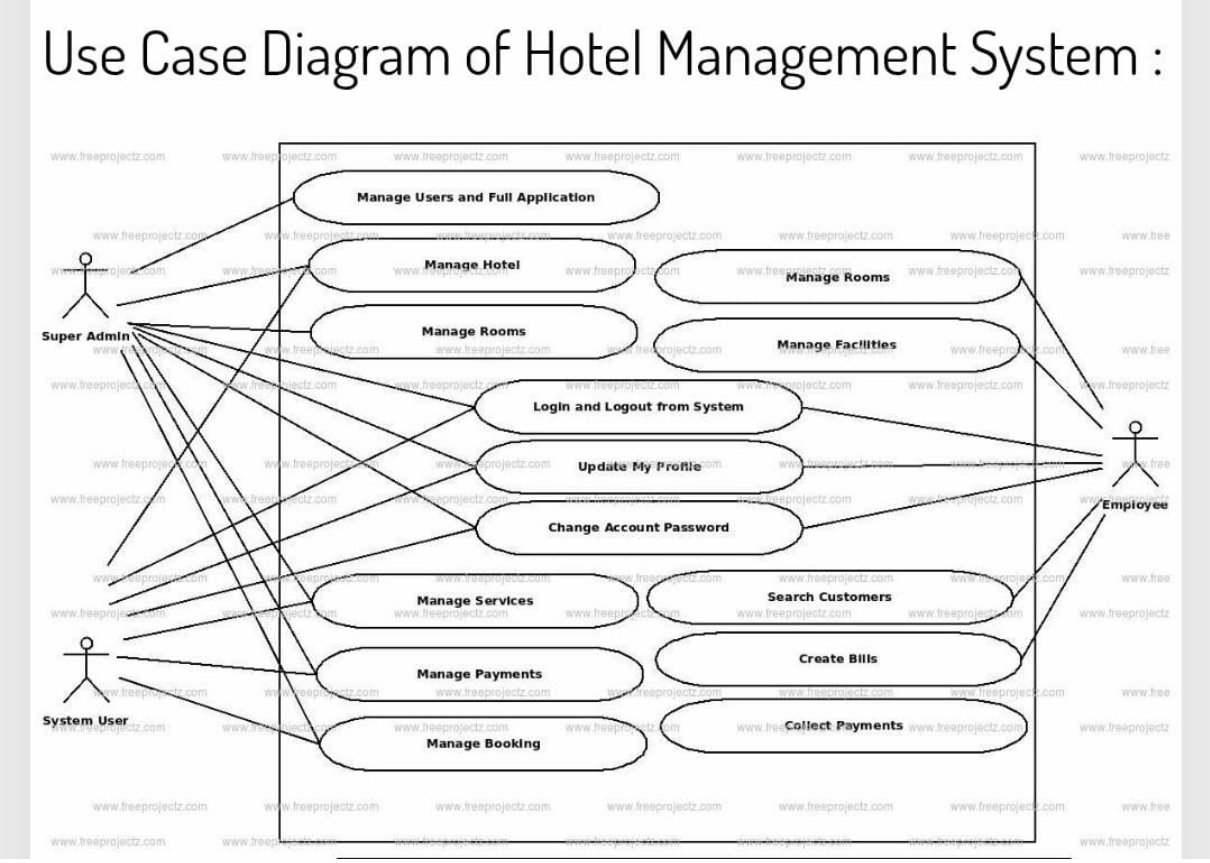
SEQUENCE DIAGRAM:



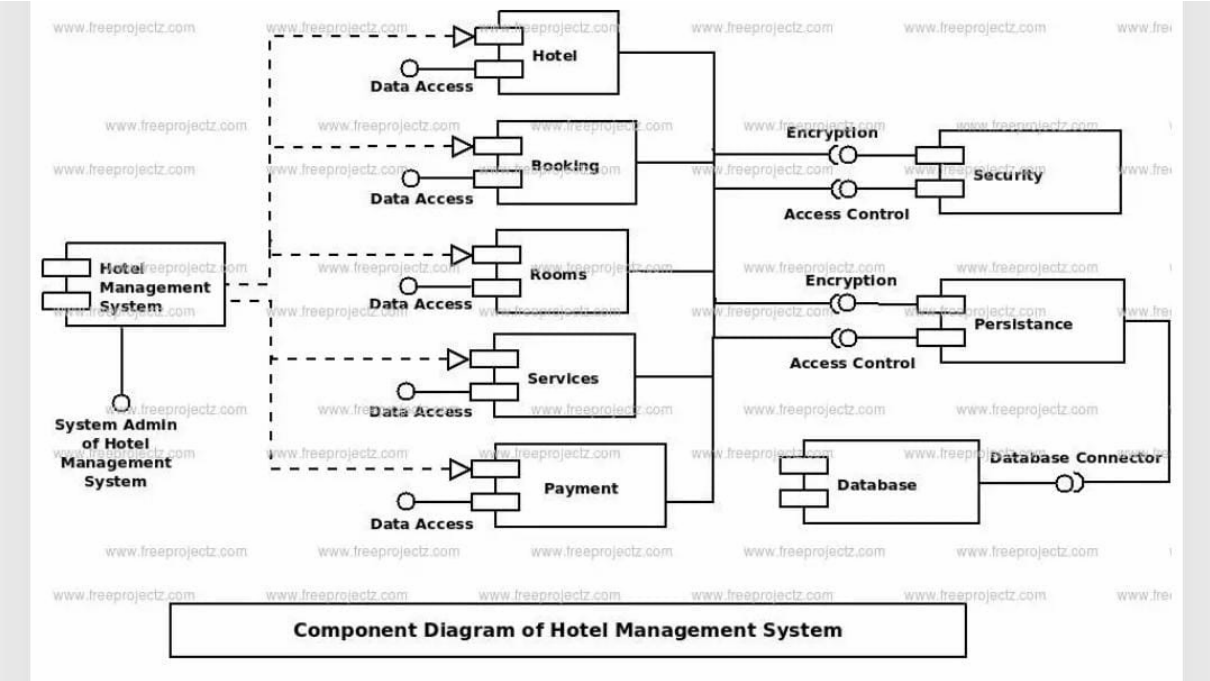
ACTIVITY DIAGRAM:



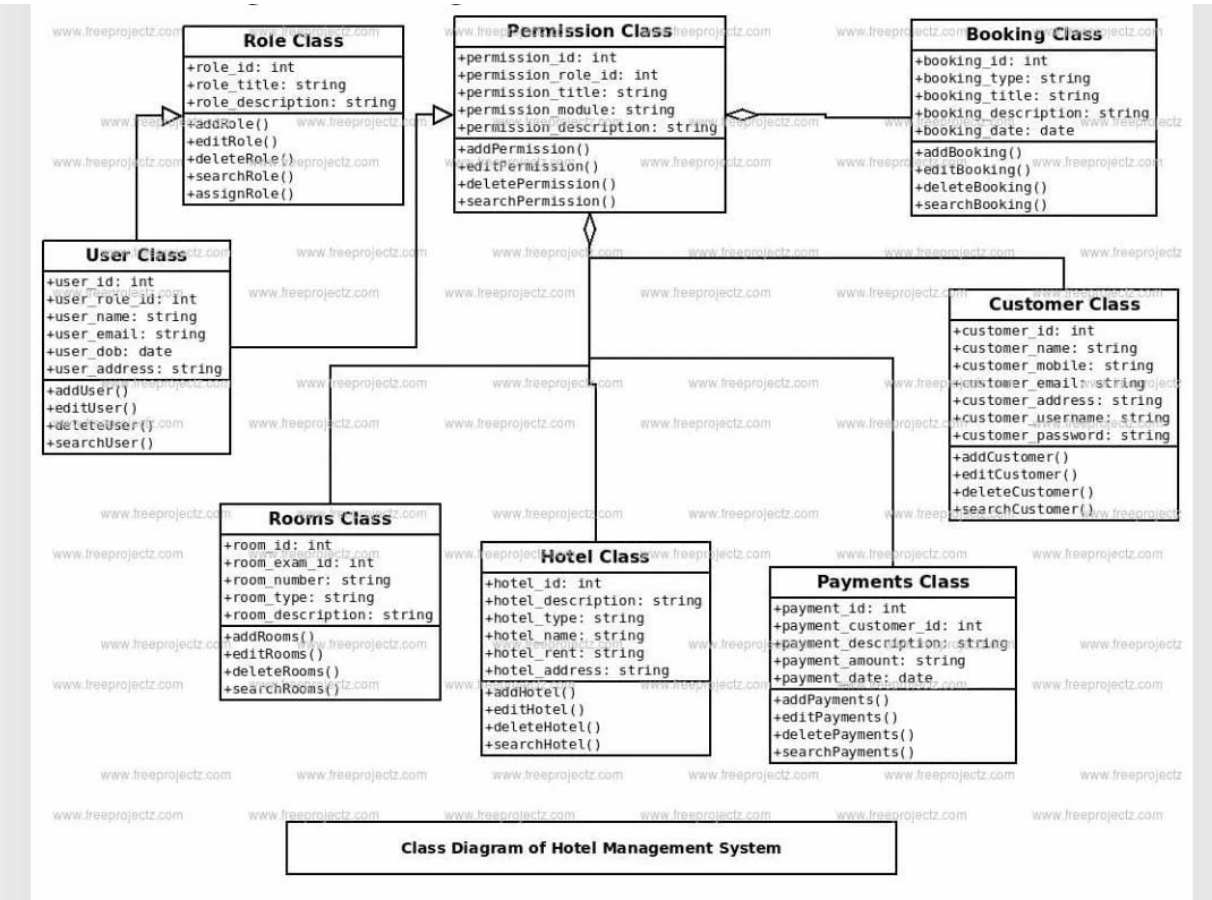
USE CASE DIAGRAM:



COMPONENT DIAGRAM :



CLASS DIAGRAM:



CODE:

1. Room.java

```
public class Room {
    private int roomNumber;
    private String roomType;
    private double price;
    private boolean isAvailable;

    public Room(int roomNumber, String roomType, double price) {
        this.roomNumber = roomNumber;
        this.roomType = roomType;
        this.price = price;
        this.isAvailable = true;
    }

    public int getRoomNumber() {
        return roomNumber;
    }
```

```
    public String getRoomType() {
        return roomType;
    }

    public double getPrice() {
        return price;
    }

    public boolean isAvailable() {
        return isAvailable;
    }

    public void setAvailable(boolean available) {
        isAvailable = available;
    }
}
```

2. Customer.java

```
public class Customer {
    private String name;
    private String id;

    public Customer(String name, String id) {
        this.name = name;
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public String getId() {
        return id;
    }
}
```

3. Booking.java

```
public class Booking {
    private Customer customer;
    private Room room;
    private String status;

    public Booking(Customer customer, Room room) {
        this.customer = customer;
        this.room = room;
        this.status = "Confirmed";
        room.setAvailable(false);
    }

    public String getBookingInfo() {
        return "Booking Status: " + status + "\nCustomer: " + customer.getName() +
                "\nRoom Number: " + room.getRoomNumber();
    }
}
```

4. HotelManagementSystem.java (Main Class)

```
import java.util.Scanner;

public class HotelManagementSystem {
    public static void main(String[] args) {
        Room room = new Room(101, "Deluxe", 2000.00);
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter customer name: ");
        String name = scanner.nextLine();
        System.out.print("Enter customer ID: ");
        String id = scanner.nextLine();

        Customer customer = new Customer(name, id);

        if (room.isAvailable()) {
            Booking booking = new Booking(customer, room);
            System.out.println("\nBooking Successful!");
            System.out.println(booking.getBookingInfo());
```

```
        } else {
            System.out.println("Room is not available.");
        }

        scanner.close();
    }
}
```

RESULT:

EXP.NO:06 **DATE: 21/03/25**

AUTOMATING THE BANKING SYSTEM

AIM :

PROBLEM ANALYSIS AND PROJECT PLANNING

INTRODUCTION:

Banking is one of the common day to day activities of life. Nowadays it is totally different from what existed a few years back, today banking has become completely computerized, new facilities such as credit cards, debit cards & ATM has been introduced. ATM stands for Automatic Teller Machine which is basically used to withdraw money from an account.

OBJECTIVES:

The objective of this software is similar to the ATM software installed in an ATM center. It first validates the pin of the ATM card, then the type of transaction is enquired and the information from the customer is validated. If it is a withdrawal the amount is asked. After the money is delivered the transaction details are updated in the database where the customer's information is stored.

SCOPE:

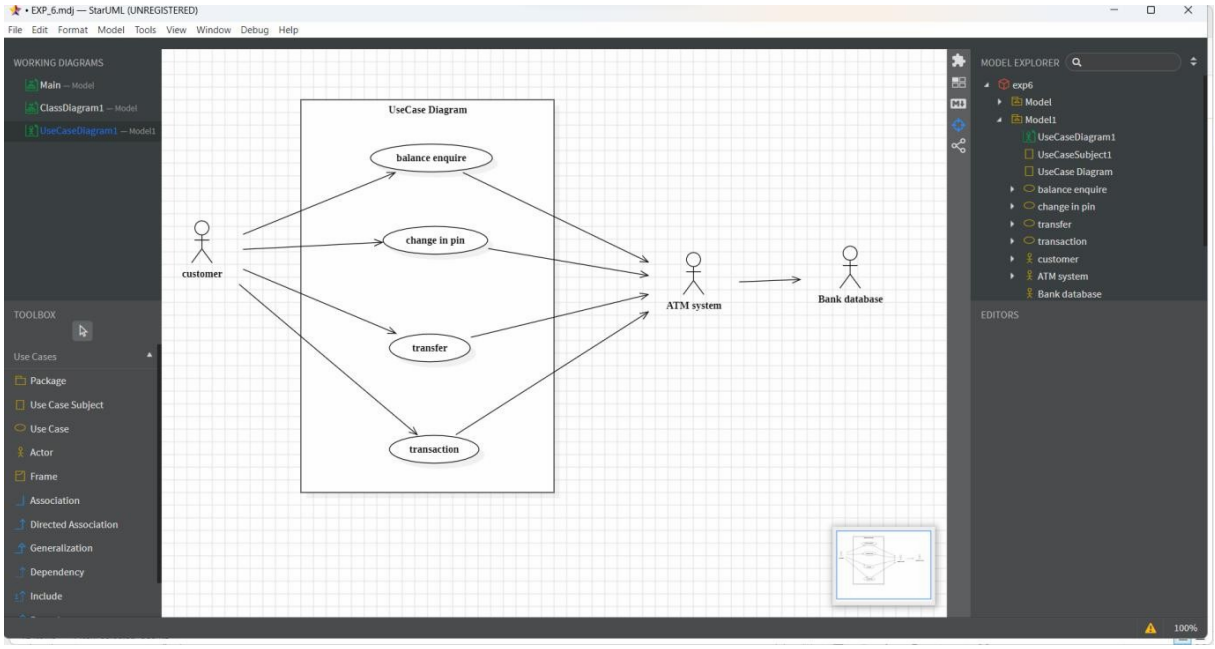
The scope of the project is to design an ATM system that will help in completely automatic banking, this software is going to be designed for withdrawal and deposit and also register the transaction in the database where the customer's information is stored.

PROBLEM STATEMENT:

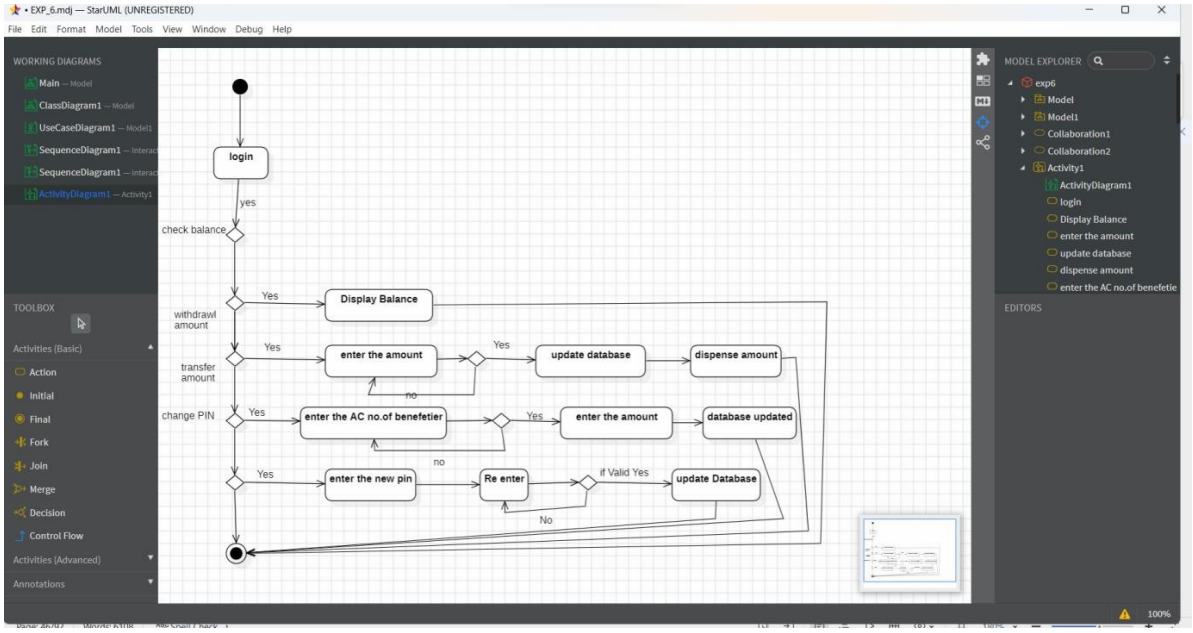
ATM banking is another type of banking where the most frequent type of transaction made is withdrawal. A user may withdraw as much as he wants until his account holds a sum greater than his withdrawal amount. ATM is completely automated and there is no necessity of the ATM center being placed at the bank itself. It can be placed in the shopping malls, airports, railway stations etc. This ATM system can use any kind of interface. But it should be user friendly and not confusing. Help manuals should be provided in case any customer has problem working with the software.

STAR UML DIAGRAMS :

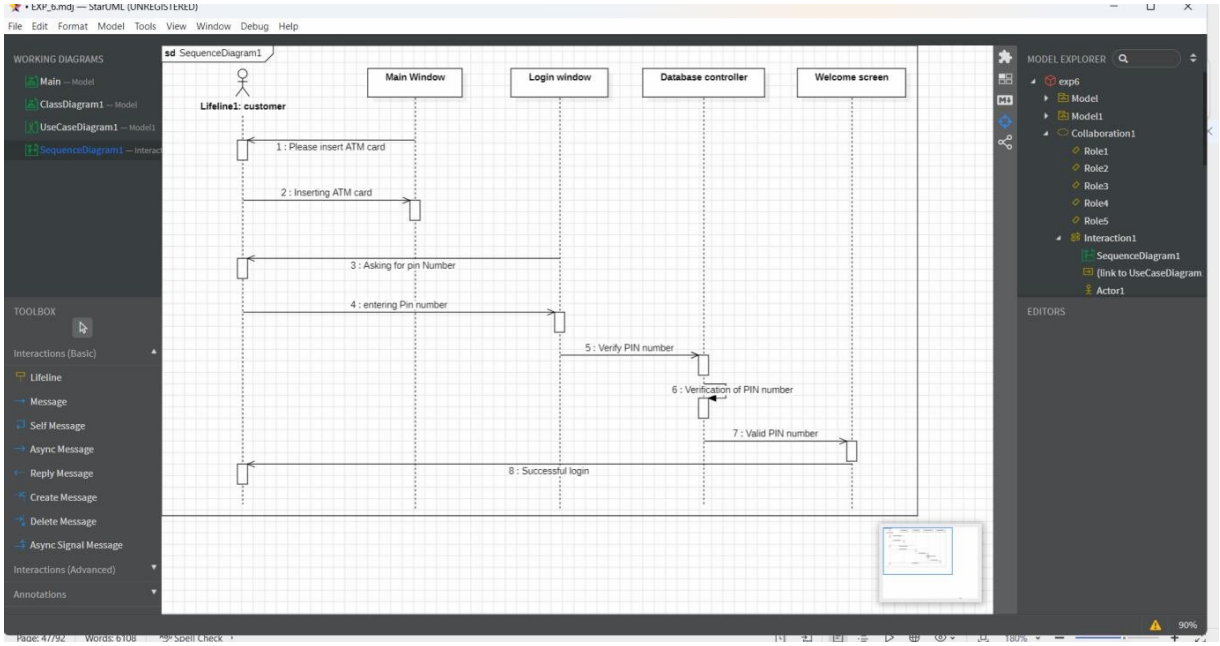
USE CASE DIAGRAM



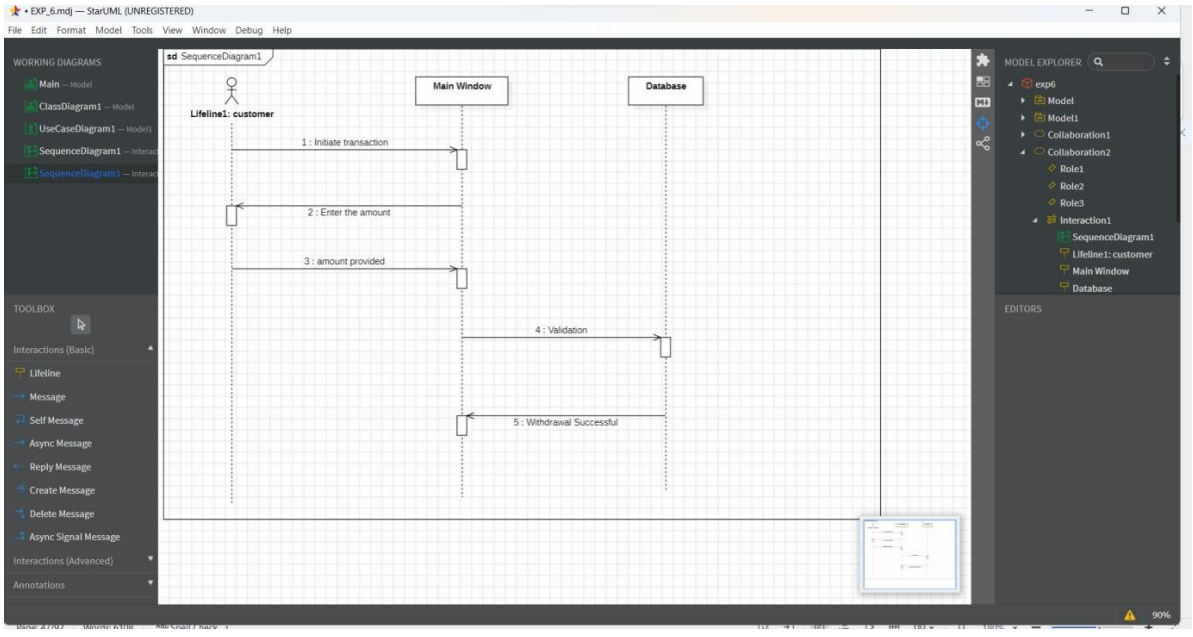
ACTIVITY DIAGRAM



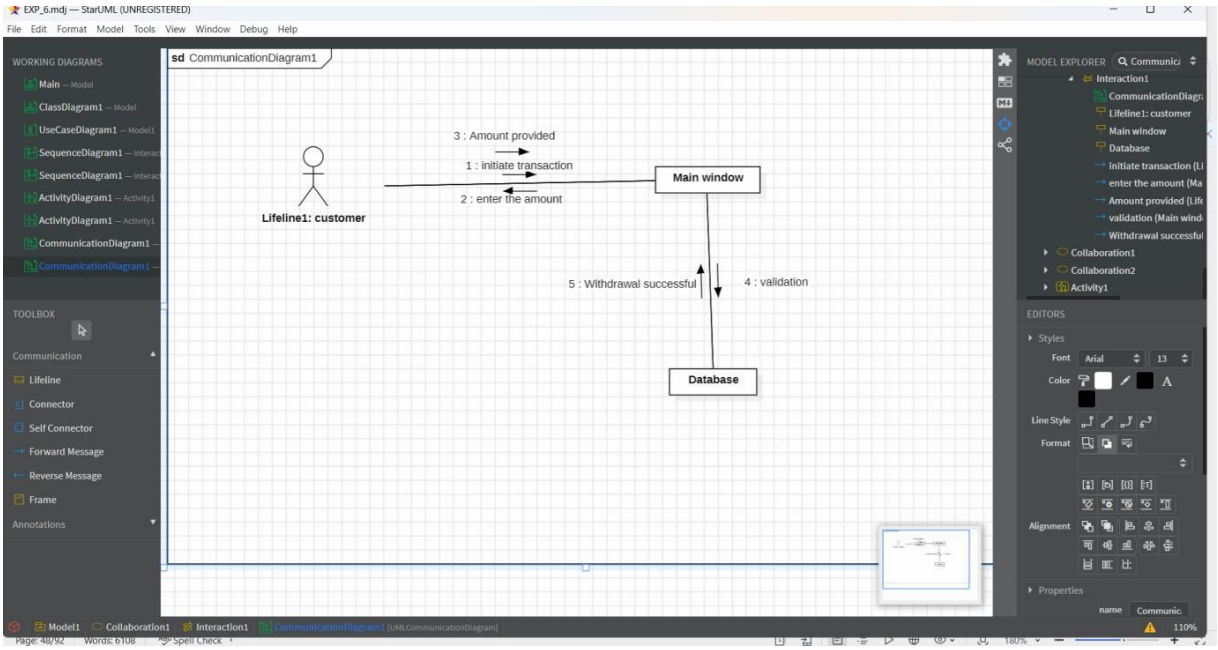
SEQUENCE DIAGRAM



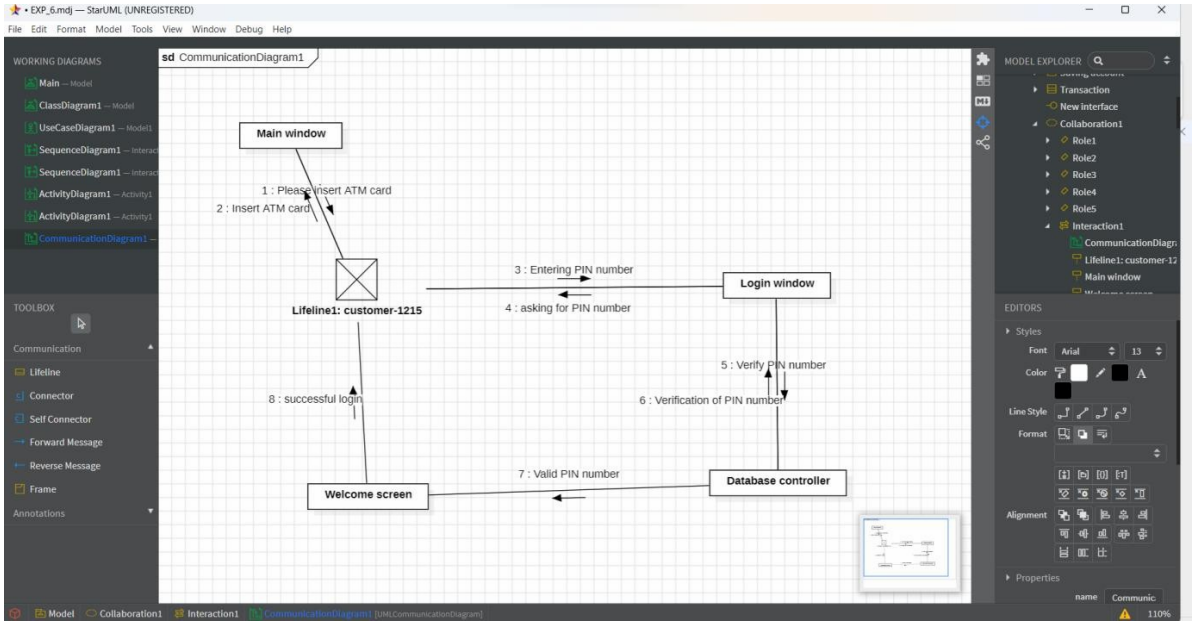
SEQUENCE DIAGRAM_02



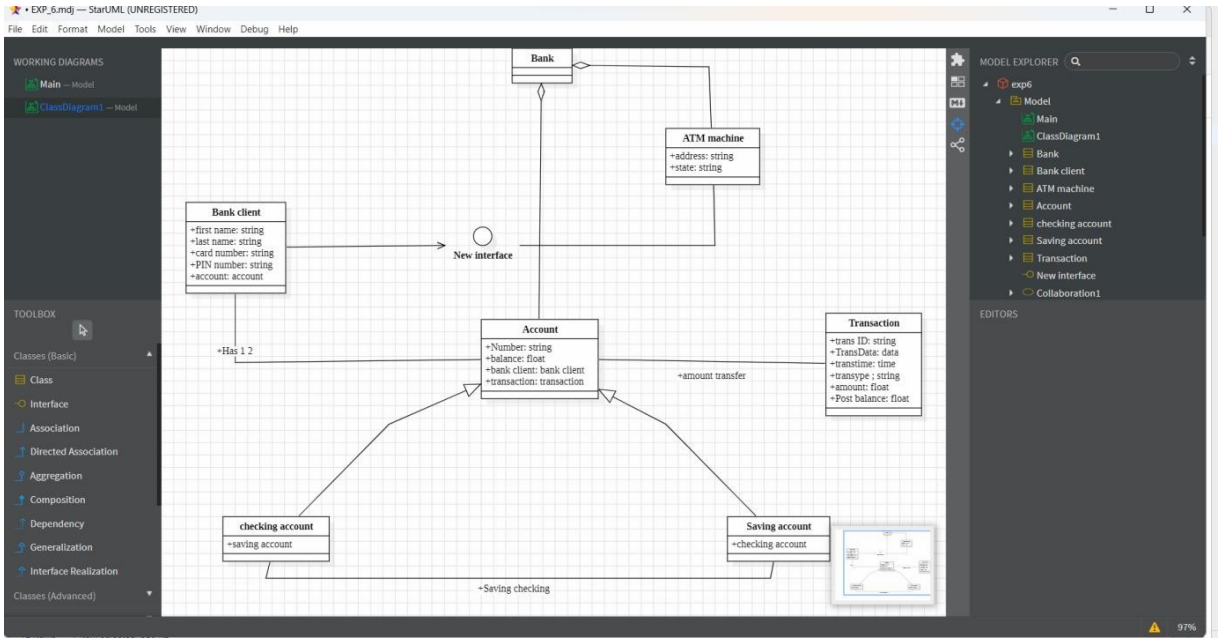
COMMUNICATION DIAGRAM



COLLABORATION DIAGRAM:



CLASS DIAGRAM



STEPS TO GENERATE CODE IN STAR UML

1. Draw the class diagram and then save it.

2. Open the saved file with star uml.

3. Then go to tools at right corner.

4. Select the code language which we can prefer.

5. Then we have the code generator in tools so give access to it.

6. Then go to tools and select the language then it will take us to another window.

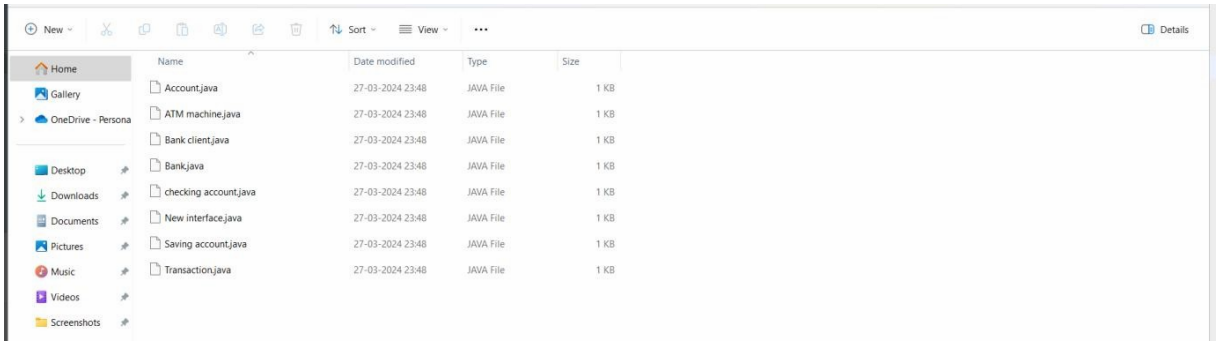
7. There select the class attribute to generate the code.

8. Then it will ask for the directory to store the code.

9. Select the directory and click on generate code.

10. Then code generated successfully

OUTPUT :



ACCOUNTS:

```
import java.util.*;

/**

 *

 */
public class Account  {
    /**
    * Default constructor
    */
   public Account () {
   }
    /**
    *
    */
   public string Number;


   /**
    *
    */
   public float balance;


   /**
    *
```

```
     */
    public bank client bank client;

    /**
     *
     */
    public transaction transaction;

}
```

ATM MACHINE:

```
import java.util.*;

/**
*
*/
public class ATM machine {

/**             Default constructor

*/

public ATM machine() {
}

/**
*
*/
public string address;

/**
*
*/
```

```
public string state;

}
```

BANK CLIENT:

```
import java.util.*;

/**
 *
 */
public class Bank client  {

    /**
     * Default constructor
     */
    public Bank client () {
    }

    /**
     *
     */
    public string first name;

    /**
     *
     */
    public string last name;

    /**
     *
     */
    public string card number;
```

```
    /**
     *
     */
    public string PIN number;

    /**
     *
     */
    public account account;

}
```

BANK:

```
import java.util.*;

/**
 *
 */
public class Bank  {

    /**
     * Default constructor
     */
    public Bank () {
    }

}
```

CHECKING ACCOUNT:

```
import java.util.*;
```

```
/**
 *
 */
public class checking account  extends Account  {

    /**
     * Default constructor
     */
    public checking account () {
    }

    /**
     *
     */
    public void saving account;

}
```

NEW INTERFACE:

```
import java.util.*;

/**
 *
 */
public interface New interface {

}
```

SAVING ACCOUNT:

```
import java.util.*;
/**
 *
```

```
 */
public class Saving account  extends Account  {
     /**
     * Default constructor
     */
    public Saving account () {
    }
     /**
     *
     */
    public void checking account;

}
```

TRANSACTIONS:

```
import java.util.*;
/**
 *
 */
public class Transaction {
      /**
     * Default constructor
     */
    public Transaction() {
    }
     /**
     *
     */
 public string trans ID;
     /**
     *
     */
```

```
    public data TransData;
     /**
     *
     */
    public time transtime;
     /**
     *
     */
    public void transype ; string;
    /**
     *
     */
    public float amount;
     /**
     *
     */
    public float Post balance;
}
```

RESULT:

EXP.NO:08 **DATE:04/04/25**

LIBRARY MANAGEMENT SYSTEM

AIM:

INTRODUCTION:

To design a library management system using IBM Rational Rose. Implement this system using Visual. Basic Language with Oracle/SQL.

OBJECTIVE:

Recently the importance of library management system within the service context of libraries has been unchallenged. Students check the list of books available and borrow the books if the book is a borrow book otherwise it is of waste for the student to come to the library to come to check for the books if the student doesn't get the book. Then the librarian checks the student id and allows the member to check out the book and the librarian then updates the member database and also the books database.

SCOPE:

This system would be used by members who may be students or professors of that University to check the availability of the books and borrow the books, and by the librarian to update the databases. The purpose of this document is to analyze and elaborate on the high level needs and features of the Online Library System. It focuses on the capabilities and facilities provided by a Library.

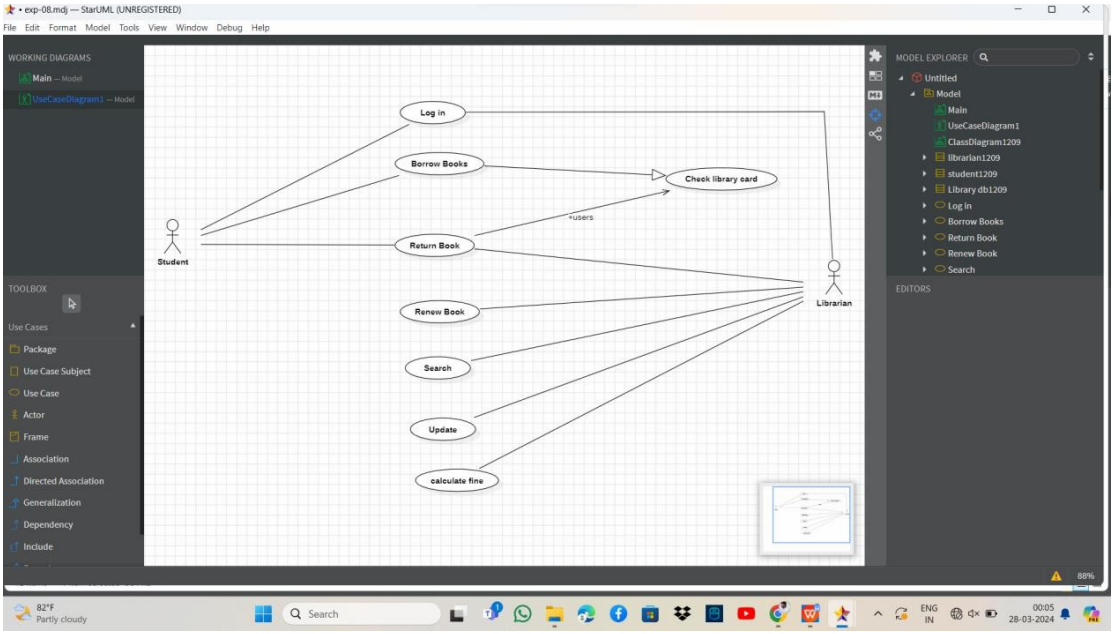
PROBLEM STATEMENT:

The Library Management System is designed & developed for a receipt and issuance of books in the library along with the student's details. A librarian can add, delete and update book status and search from the database. A user can borrow, return books, reserve books and search for books. He can also renew his loan period. If a book is overdue, the user will be fined Rs.5 each day over the due . If a book is reported lost, the user will pay the full cost of the book. The books received in the library are entered in Books Entry form and the new

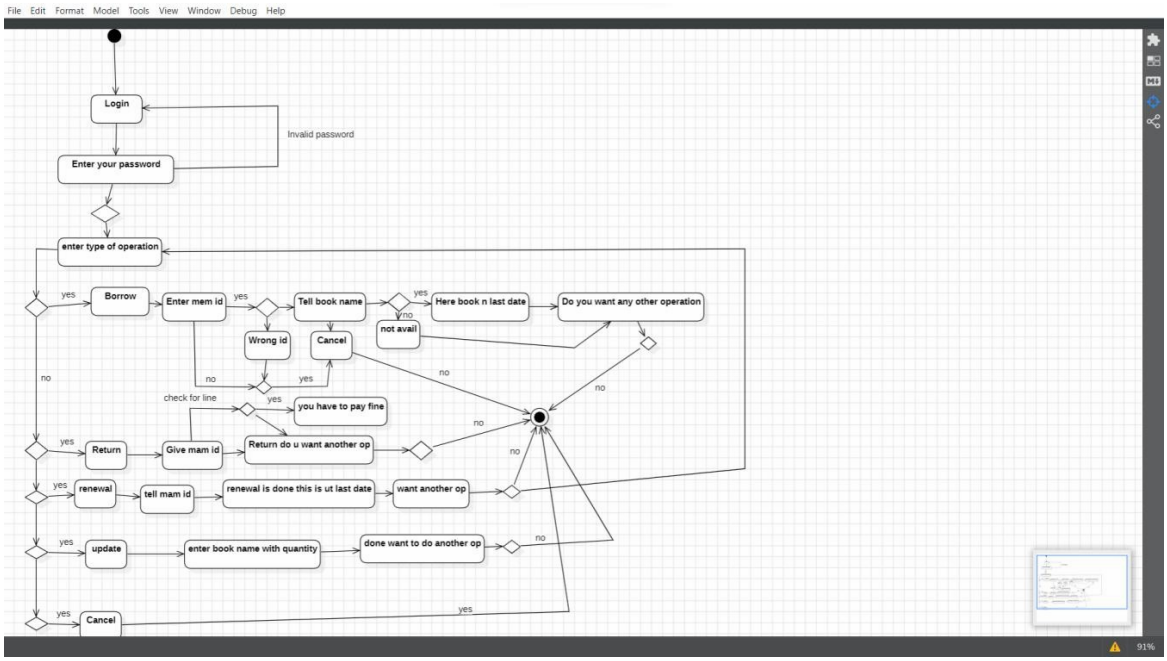
student is entered in the student entry form. When the student wants to get the desired book the same is issued on availability basis to the student.

STAR UML DIAGRAMS:

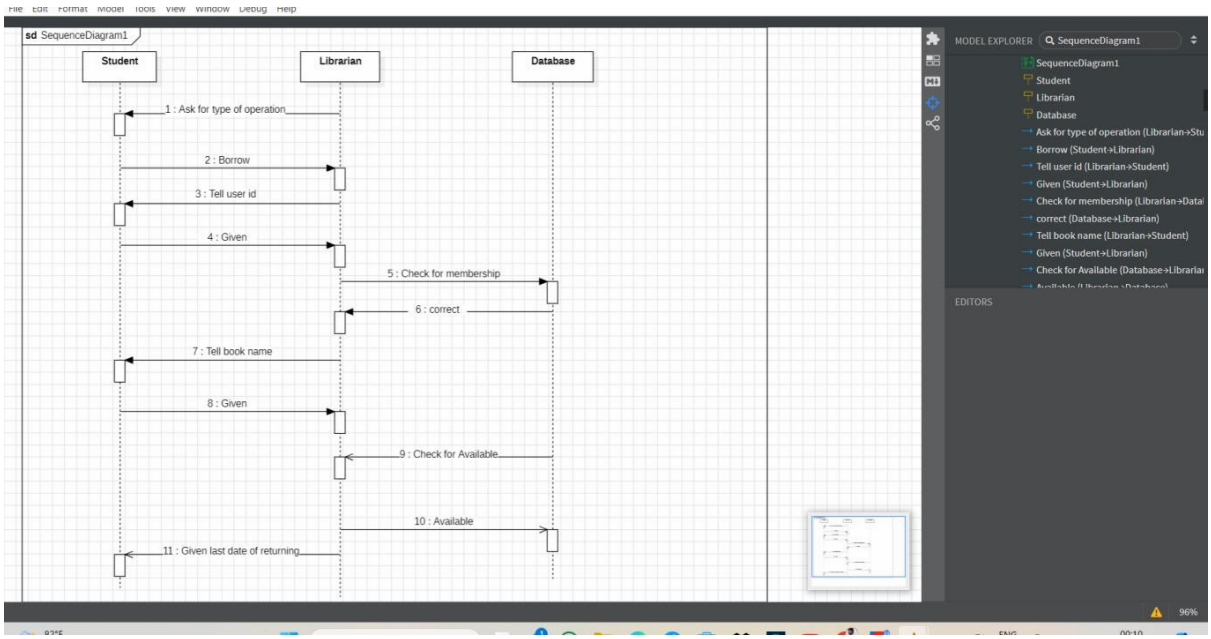
USE CASE DIAGRAM:



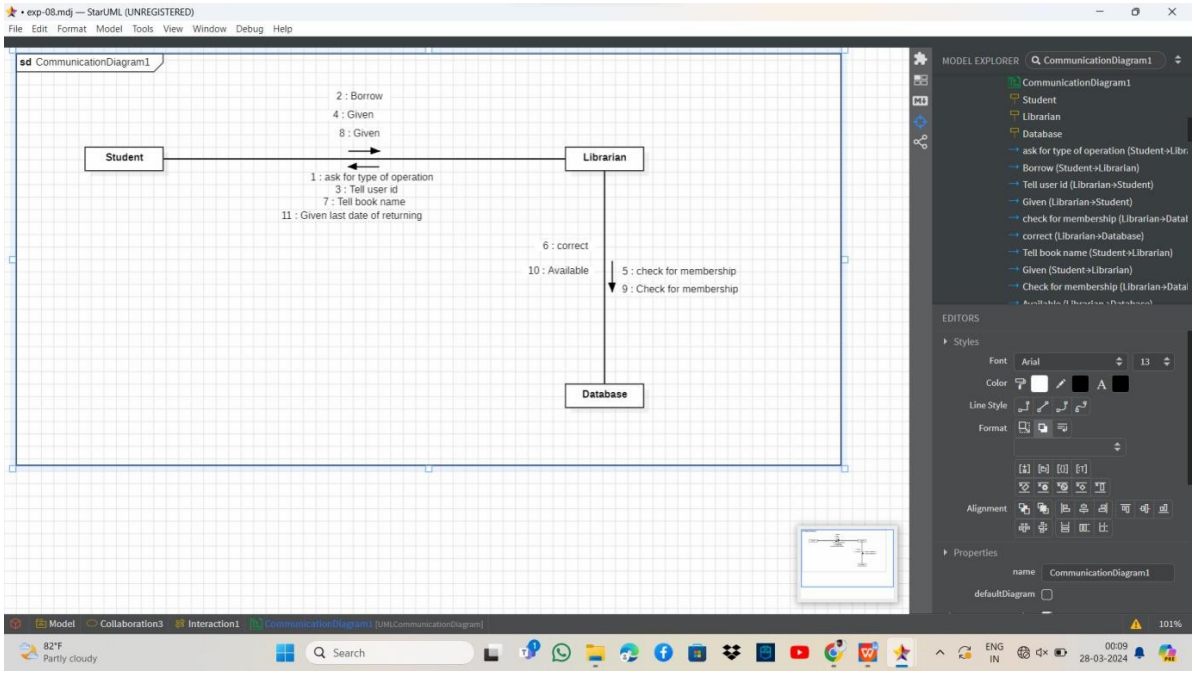
ACTIVITY DIAGRAM:



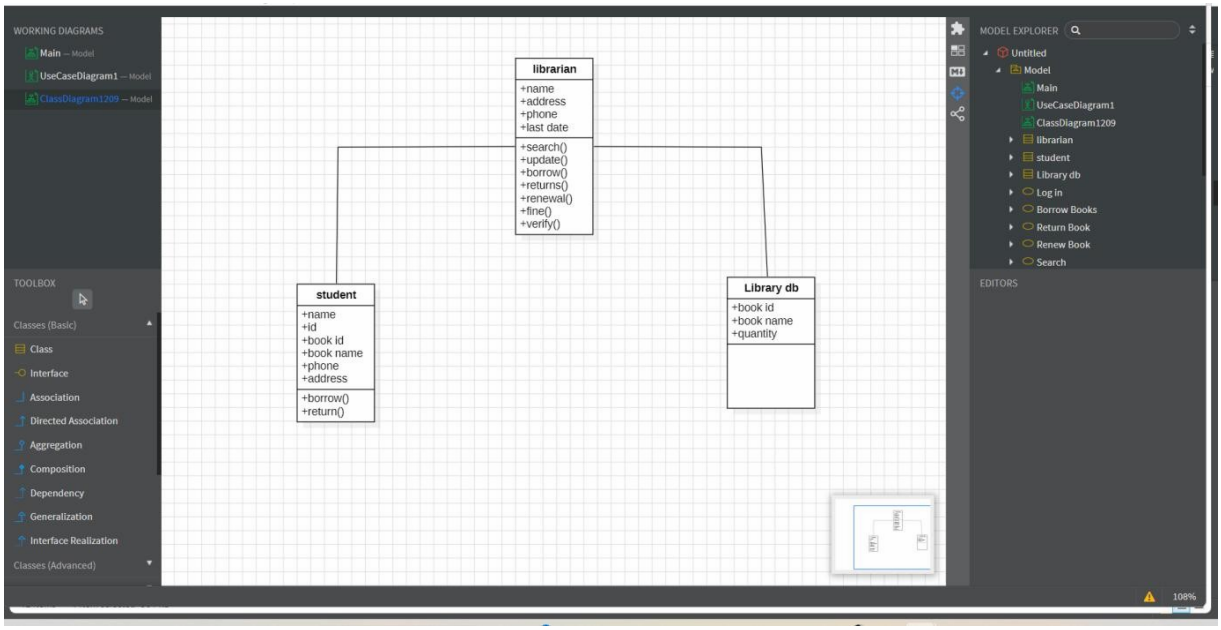
SEQUENCE DIAGRAM:



COMMUNICATION DIAGRAM:



CLASS DIAGRAM:



STEPS TO GENERATE CODE IN STAR UML

1. Draw the class diagram and then save it.

2. Open the saved file with star uml.

3. Then go to tools at right corner.

4. Select the code language which we can prefer.

5. Then we have the code generator in tools so give access to it.

6. Then go to tools and select the language then it will take us to another window.

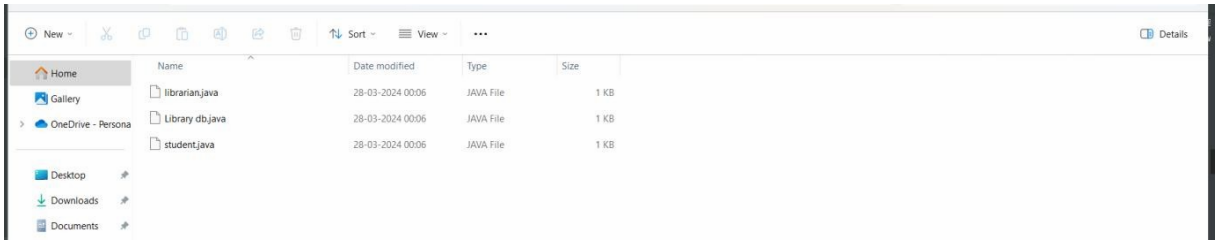
7. There select the class attribute to generate the code.

8. Then it will ask for the directory to store the code.

9. Select the directory and click on generate code.

10. Then code generated successfully

OUTPUT:



LIBRARIAN:

```
import java.util.*;

/****/public class librarian {

 /**

     * Default constructor

     */

    public librarian () {

    }

    /***

     */

    public void name;

    /**

 **/

    public void address;

 /****/

    public void phone;

    /****/

    public void last date;

    /****/

    public void search() {
```

```
    // TODO implement here
    }
    /****/
    public void update() {
    // TODO implement here
    }
    /****/
    public void borrow() {
        // TODO implement here
    }
/****/
public void returns() {
        // TODO implement here
    }
   /****/
    public void renewal() {
        // TODO implement here
    }
    /****/
    public void fine() {
        // TODO implement here
    }
 /****
    public void verify() {
```

```
        // TODO implement here
    }
}
```

LIBRARY DATABASE:

```
import java.util.*;
/****/
public class Library db  {
/**
     * Default constructor
     */
    public Library db () {
    }
/*** */
    public void book id;
    /****/
    public void book name;

    /***/
    public void quantity;
}
```

STUDENT:

```
import java.util.*;

/*
public class student {

/*** Default constructor*/
/****/

public void book name;


/****/

public void phone;


/****/

public void address;

/****/

public void borrow() {

// TODO implement here

}

/****/

public void return() {

// TODO implement here

}


}
```

RESULT:

EXP.NO: 08 **DATE:04/04/25**

PASSPORT AUTOMATION SYSTEM

AIM:

OBJECTIVES:

To develop the passport automation system software using UML language. It is the interface between applicant and authority responsible for issue the passport. It aims at improving efficiency and reducing complexities.

SCOPE:

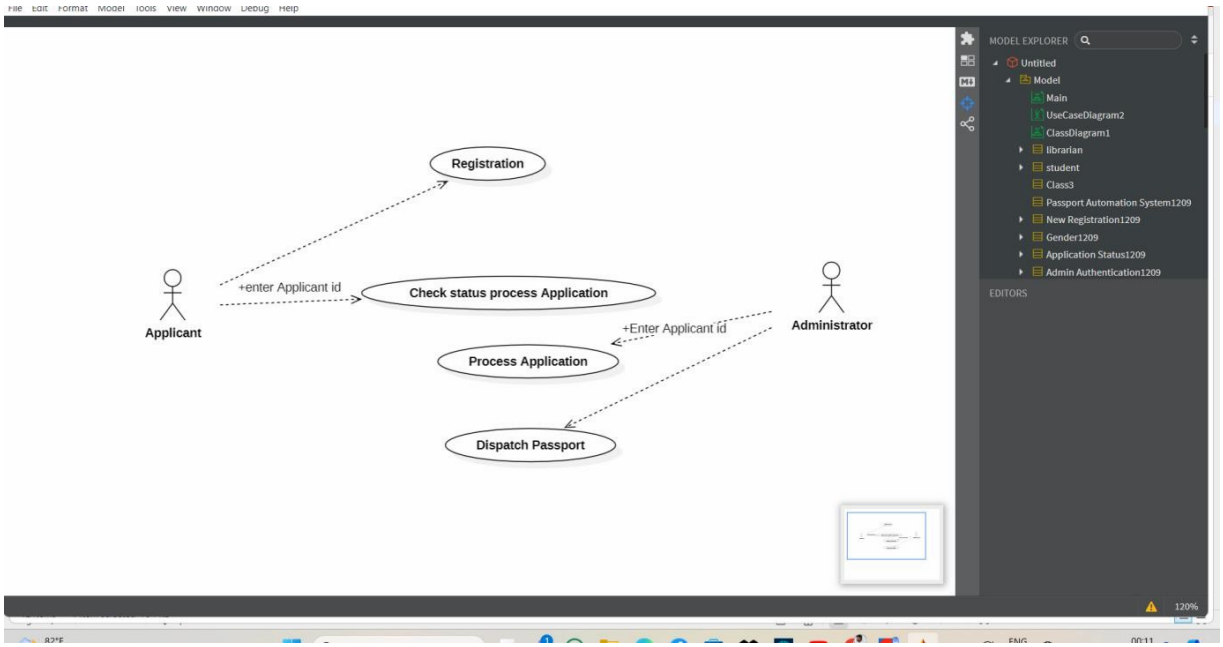
The system provides the online interface to the user where we can fill their form and personal detail with necessary proof. The authority concerned with the issue of passport can use this system to reduce is workload and process it speedy manner. It provides communication platform between administrator and applicant. To transfer the data between passport authority and local police verifying the applicant's information.

PROBLEM STATEMENT:

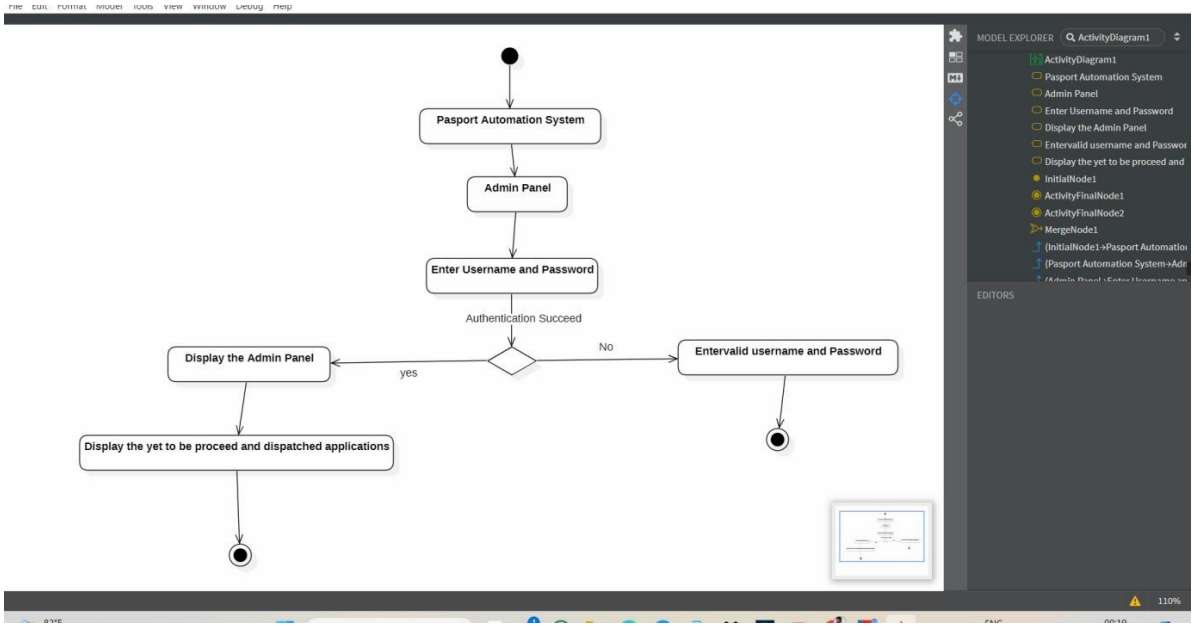
My project title is passport automation system. In this project we can develop the modules such as login, fill the form with necessary proof, verifying the applicant's information, validity checking and issue the passport for that particular applicant. In this login module, we can perform that that perform that enters into the login website for the different actors, and then fill for the can be done by the primary and verification, checking and issuing the passport can be done for the supporting actor.

STAR UML DIAGRAMS:

USE CASE DIAGRAM:

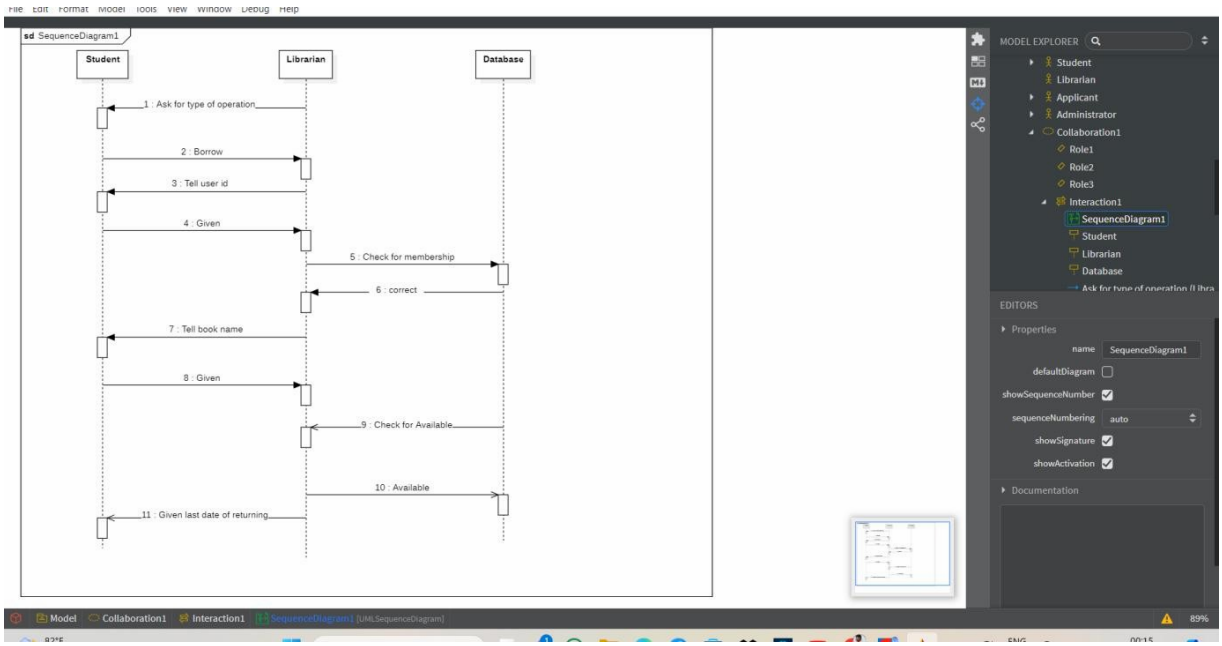


ACTIVITY DIAGRAM:

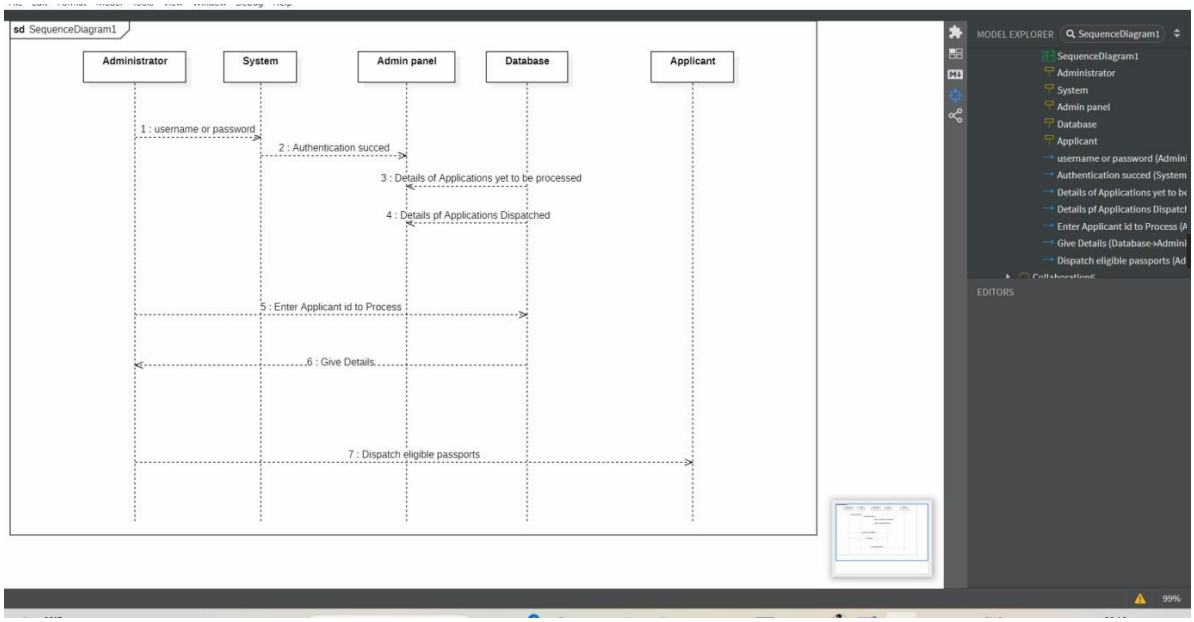


SEQUENCE DIAGRAM:

NEW REGISTRATION

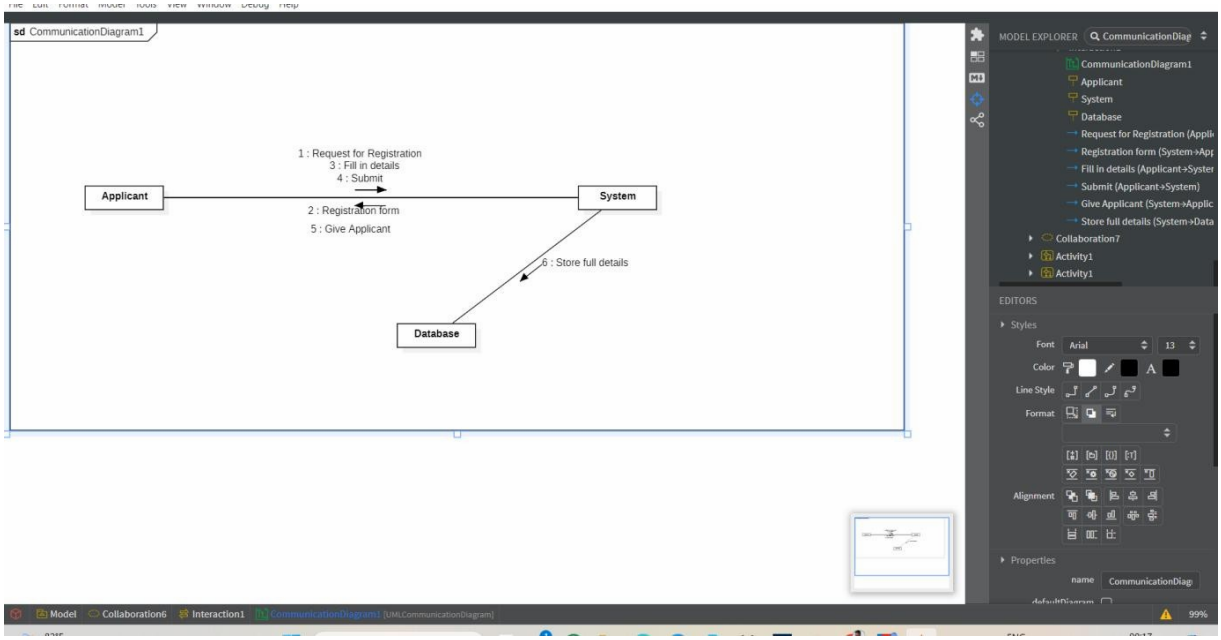


ADMIN PANEL

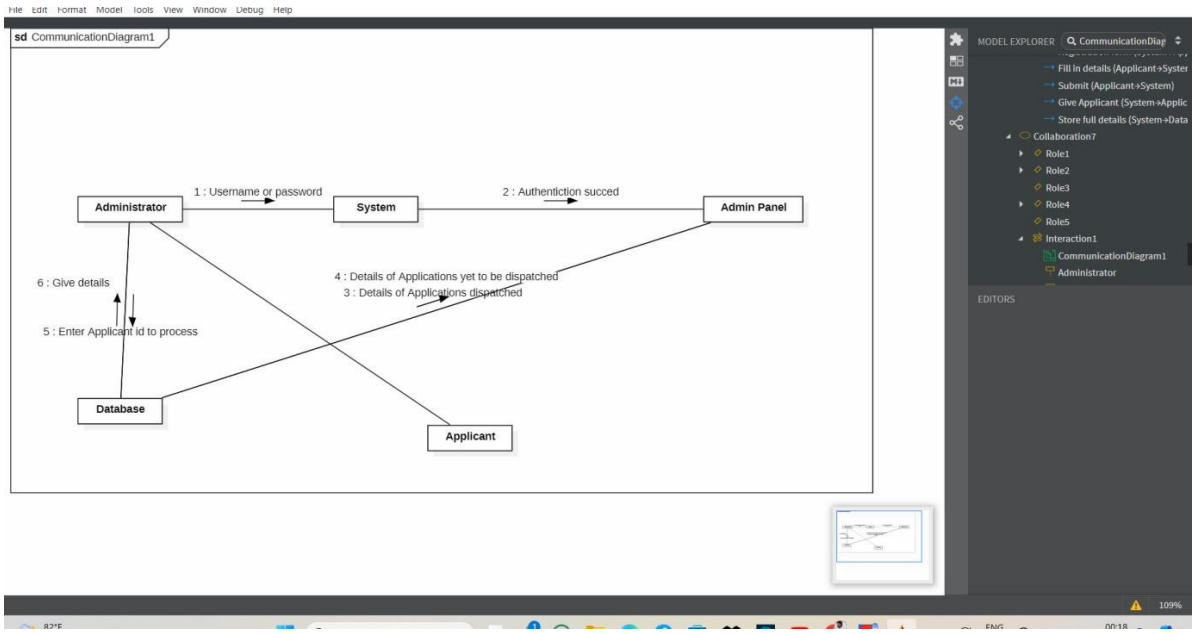


COMMUNICATION DIAGRAM:

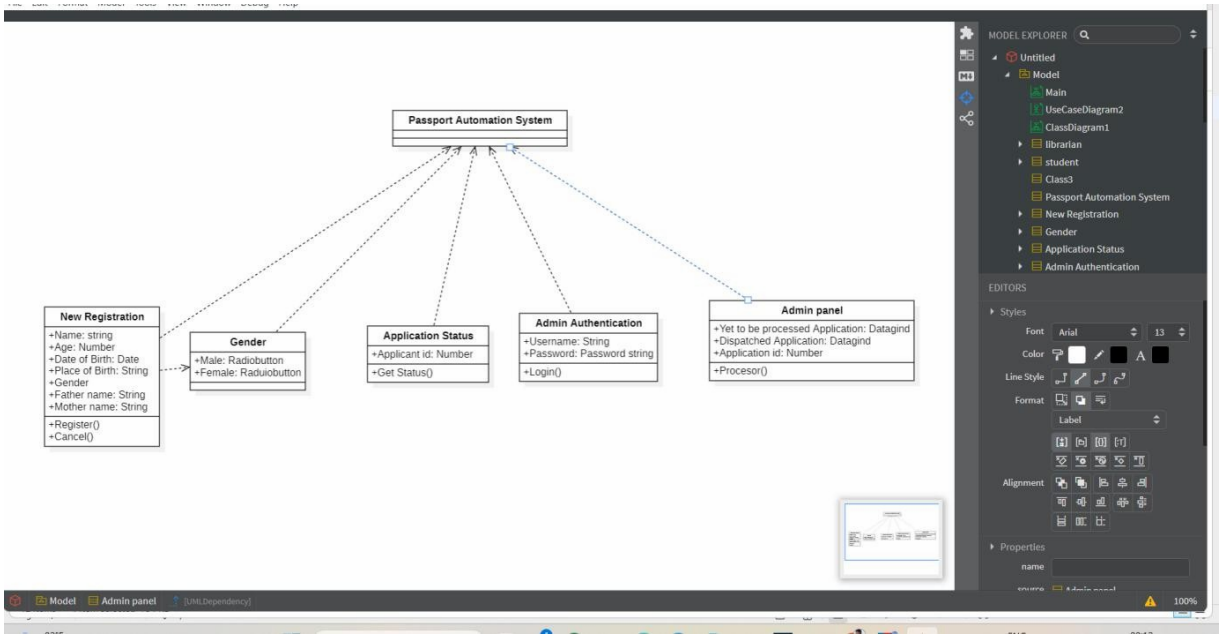
NEW REGISTRATION



ADMIN PANEL



CLASS DIAGRAM:



ALGORITHM:

1. Draw the class diagram and then save it.

2. Open the saved file with star uml.

3. Then go to tools at right corner.

4. Select the code language which we can prefer.

5. Then we have the code generator in tools so give access to it.

6. Then go to tools and select the language then it will take us to another window.

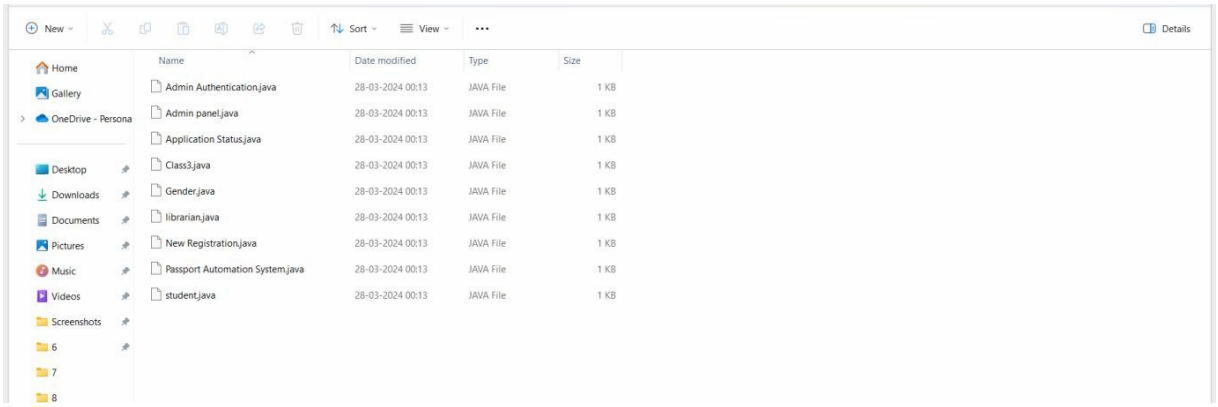
7. There select the class attribute to generate the code.

8. Then it will ask for the directory to store the code.

9. Select the directory and click on generate code.

10. Then code generated successfully.

OUTPUT:



ADMIN PANEL:

```
import java.util.*;
/**
 *
 */
public class Admin  panel {
    /**
     * Default constructor
     */
    public Admin  panel() {
    }

    /**
     *
     */
    public Datagind yet to be processed application;

    /**
     *
     */
```

```
    public Datagind- Dispatched Application;

  /**
     *
     */
    public Number Application id;

    /**
     *
     */
    public void Process() {
        // TODO implement here
    }
}
```

ADMIN AUTHENTICATION:

```
import java.util.*;
/**
 *
 */
public class Admin Authentication {

    /**
     * Default constructor
     */
    public Admin Authentication() {
    }

    /**
     *
     */
```

```
    public string Username;

    /**
     *
     */
    public password string password;
    /**
     *
     */
    public void Login() {
        // TODO implement here
    }
}
```

APPLICATION STATUS:

```
import java.util.*;
/**
 *
 */
public class Application Status {

    /**
     * Default constructor
     */
    public Application Status() {
    }

    /**
     *
     */
    public Number Application id;
```

```
    /**
     *
     */
    public void Attribute2;
     /**
     *
     */
    public void Get Status() {
        // TODO implement here
    }
}
```

GENDER:

```
import java.util.*;
/**
 *
 */
public class Gender {
     /**
     * Default constructor
     */
    public Gender() {
    }
     /**
     *
     */
    public Radiobutton Male;
     /**
     *
     */
    public Radio button Female;
     /**
```

```
     *
     */
    public void Attribute3;
}
```

NEW REGISTRATION:

```
import java.util.*;
/**
 *
 */
public class New Registration {
     /**
     * Default constructor
     */
    public New Registration() {
    }

    /**
     *
     */
    public string Name;

    /**
     *
     */
    public Number Age;

    /**
     *
     */
    public string Date of Birth;
```

```
/**
 *
 */
public void Gender;

/**
 *
 */
public string Father name;

/**
 *
 */
public string Mother name;

/**
 *
 */
public void Attribute7;

/**
 *
 */
public void Attribute8;

/**
 *
 */
public void Register() {
    // TODO implement here
}
```

```
    /**
     *
     */
    public void cancel() {
        // TODO implement here
    }
}
```

PASSPORT AUTOMATION SYSTEM:

```
import java.util.*;
/**
 *
 */
public class Passport Automation System {
    /**
     * Default constructor
     */
    public Passport Automation System() {
    }
}
```

RESULT:

EXP.NO:09 **DATE:11/04/25**

E-TICKETING

AIM :

INTRODUCTION:

The manual system of ticket reservation takes more time and the number of reservations per day is limited. To increase the efficiency of the process, we go for online ticket reservation system. This system supports online ticket booking..

OBJECTIVE:

If the entire process of reservation is done in a manual manner then it would takes several months for reservation to reach the applicant. Considering the fact that the number of passenger is increasing every year, an Automated System becomes essential to meet the demand. So this system uses several programming and database techniques to elucidate the work involved in this process. As this is a matter of National Security, the system has been carefully verified and validated in order to satisfy it

.SCOPE:

• The System provides an online interface to the user where they can fill in their personal details and submit the necessary documents (may be by scanning).

• The authority concerned with the issue of railway can use this system to reduce his workload and process the application in a speedy manner.

• Provide a communication platform between the passenger and the administrator.

• Passenger will come to know their status of application and the date in which they must subject themselves for manual document verification.

PROBLEM STATEMENT:

The current airline reservation process is often fragmented, involving manual operations, third-party travel agencies, and disjointed data systems. These inefficiencies result in long wait times, booking errors, difficulty in tracking tickets, and poor customer experience. Additionally, airlines face challenges in managing seat availability, dynamic pricing, and customer data securely and efficiently.

USE-CASE NAME: CHECK STATUS The passenger can view the status of the reserved tickets. So the passenger can confirm his/her travel.

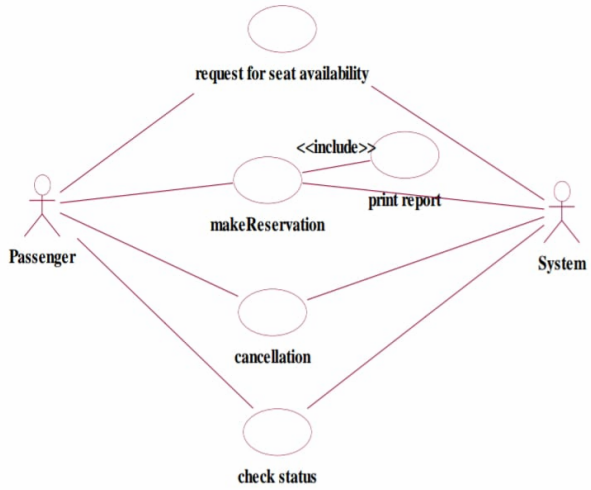
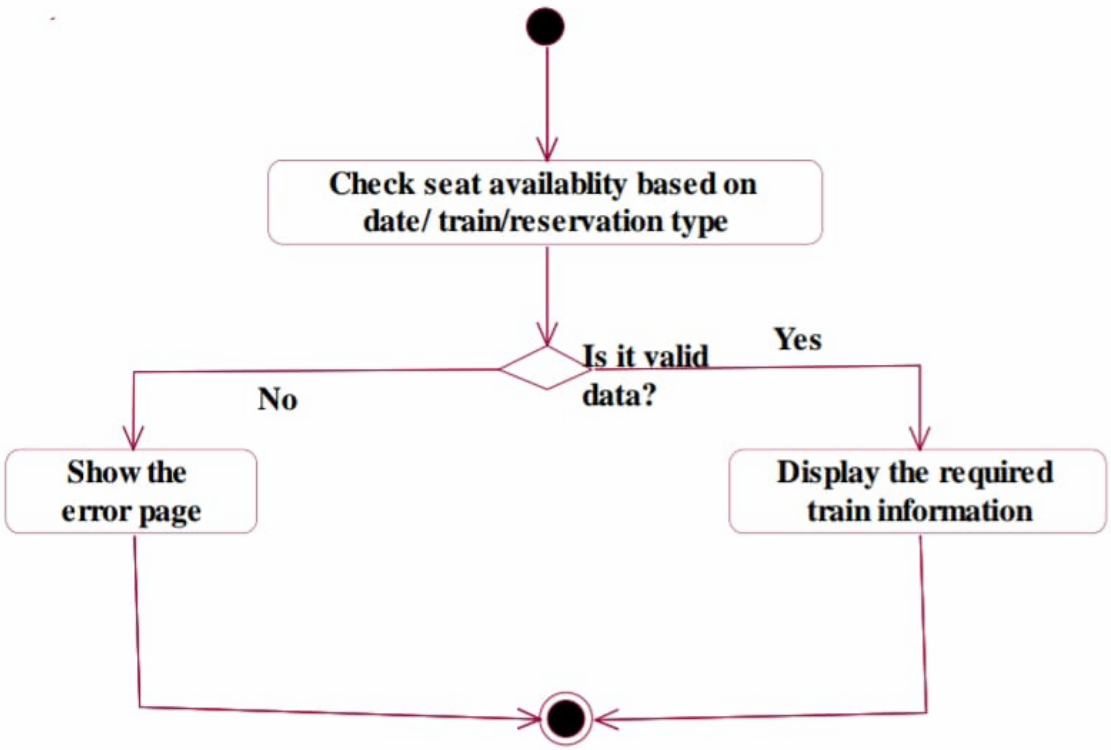
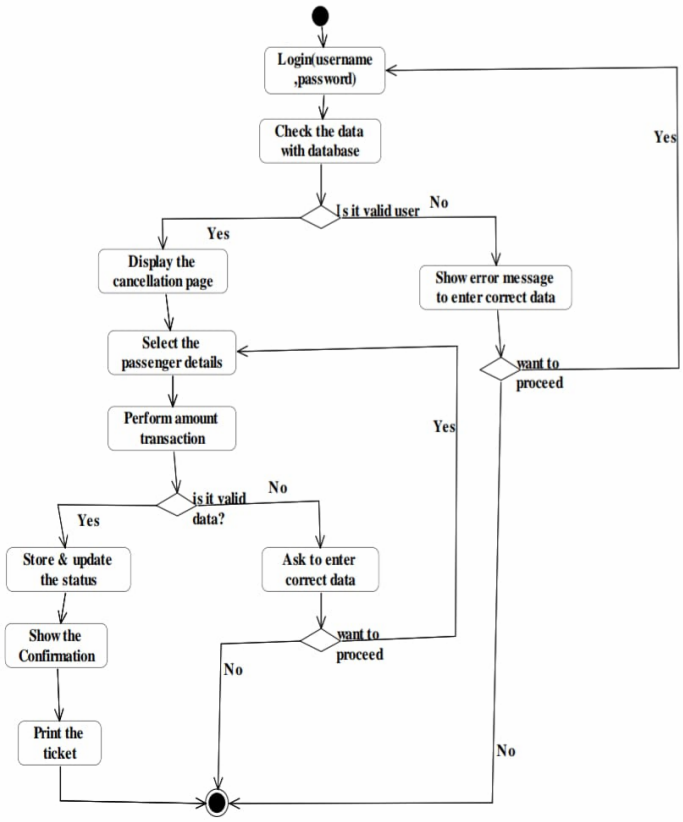


Fig.3.1 USE-CASE DIAGRAM FOR AIRLINE RESERVATION

ACTIVITY DIAGRAM:

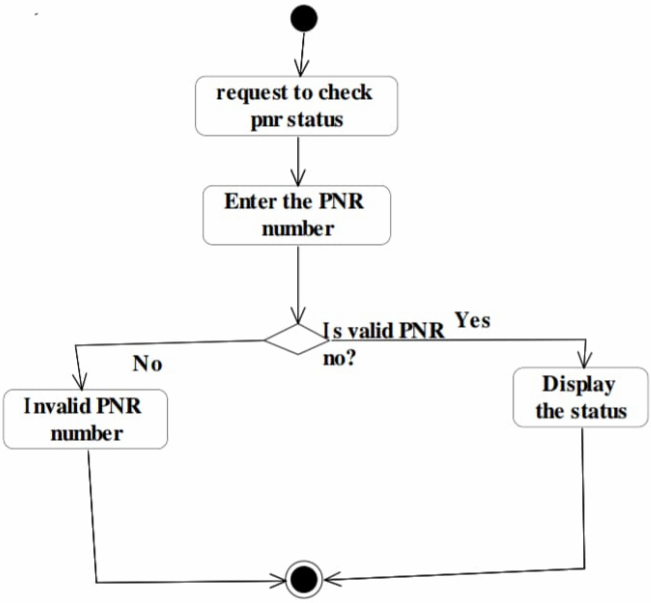


ACTIVITY DIAGRAM:(TICKET CANCELLATION)

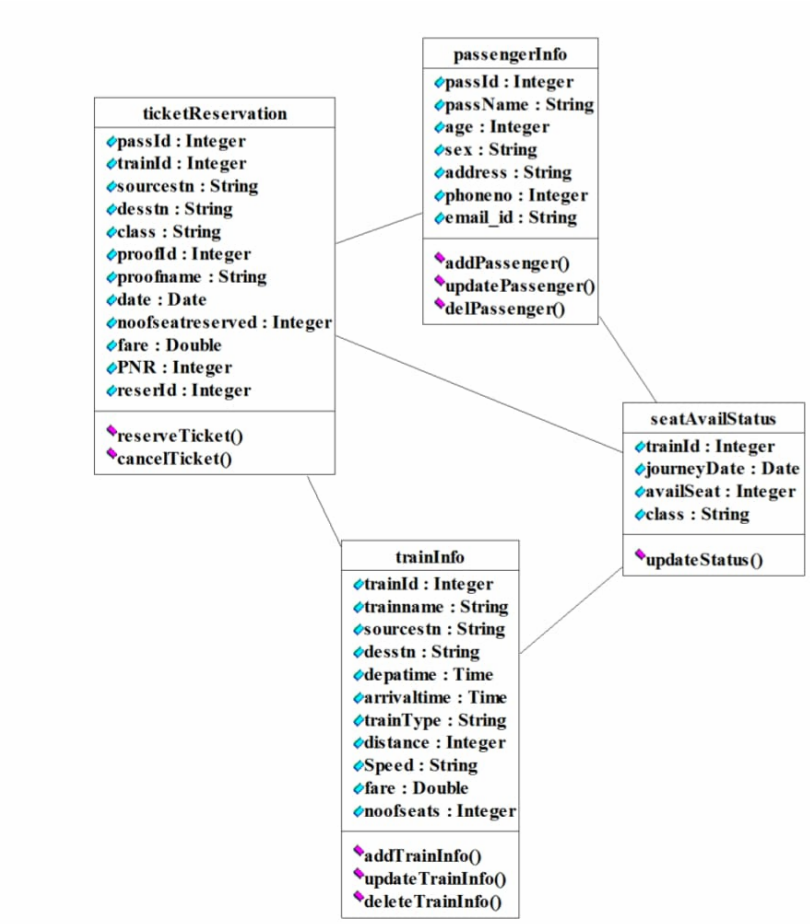


ACTIVITY DIAGRAM:

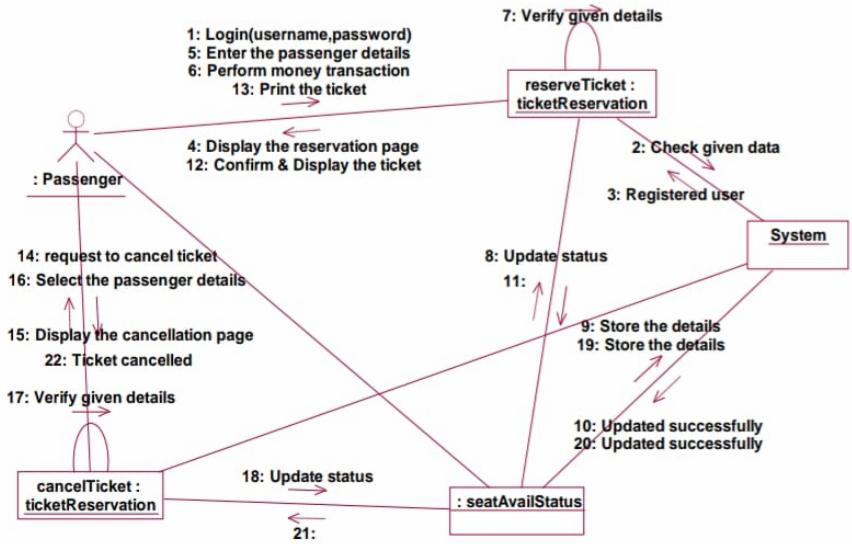
CHECK STATUS



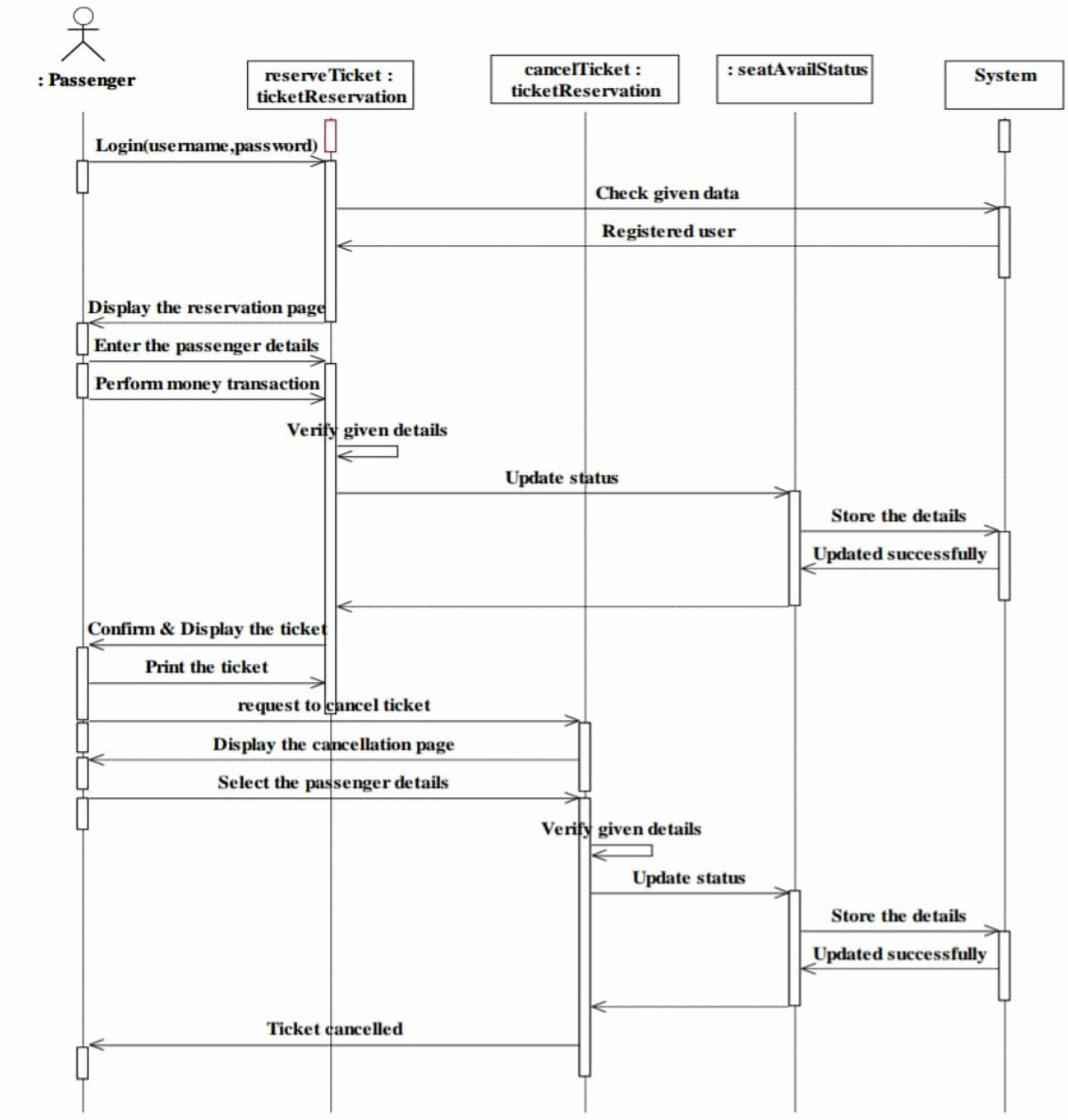
CLASS DIAGRAM:



COLLABORATION DIAGRAM:



SEQUENCE DIAGRAM:



IMPLEMENTATION OF DOMAIN OBJECTS LAYER AND TECHNICAL SERVICE LAYER

CODE:

//Source file: E:\\sugan\\passengerInfo.java

```
public class passengerInfo
 {
```

```
private integer passId;
private String PassName;
private integer age;
private string sex;
private integer phoneno;
private String emailid

/**
@roseuid 5121D2AB01D4
*/ public passengerInfo()
{

 }
/**
 @roseuid 51188FBF0203
*/
public void addPassenger()
 { }
/**
@roseuid 51188FD00148
*/
public void updatePassenger()
{ }
/**
@roseuid 51188FD900FA
*/
 public void delPassenger()
 { }
}
/**
 void passengerinfo.updatepass()
{ }
 void passengerinfo.addpass()
{ }
void passengerinfo.delpass()
{ }
 passengerinfo.passengerinfo()
*/
```

RESULT :